Pontificia Universidad Católica del Perú
Facultad de Ciencias e Ingeniería
Departamento Académico de Ingeniería

1INF02 APRENDIZAJE DE MÁQUINA

Modelos de regresión y clasificación

Prof. Rodrigo Uribe

Organización de las Clases

(10 min) Preguntas y consultas.

(30 min) Respuesta *Vibe Quest.*

(45 min) Regresión y descenso de gradiente.



(10 min) Break

(10 min) Dinámica grupal

(45 min) Clasificación binaria y logística. Problemas matemáticos.

(10 min) Preguntas *Vibe Quest.*

Respuestas Vibe Quest

- El Teorema Central del Límite promete que la normalidad emerge de sumar cualquier distribución. ¿Qué tan rápido ocurre esto? Experimenta con distribuciones extrañas y encuentra cuántas muestras necesitas antes de que aparezca la campana gaussiana.
- Frecuentistas y bayesianos cuantifican la incertidumbre de formas filosóficamente opuestas. ¿Puedes demostrar computacionalmente la diferencia entre un intervalo de confianza del 95% y un intervalo de credibilidad del 95%?
- En aprendizaje supervisado, un modelo demasiado simple subajusta y uno demasiado complejo sobreajusta. Explora cómo diferentes niveles de complejidad del modelo (desde una línea recta hasta un polinomio de grado alto) afectan la capacidad de generalización, y encuentra donde el error total se minimiza. Haz diversos ejemplos con modelos “grandes”.

Clase 1:

¿Qué es aprendizaje de máquina?

(Computadoras aprendiendo de datos automáticamente)

Clase 2:

¿Cómo se toman decisiones (modelado) con datos?

Clase 3:

¿Cómo cambio el análisis y modelado de datos?

Clase 4:

¿Cómo los modelos aprenden de equivocarse?



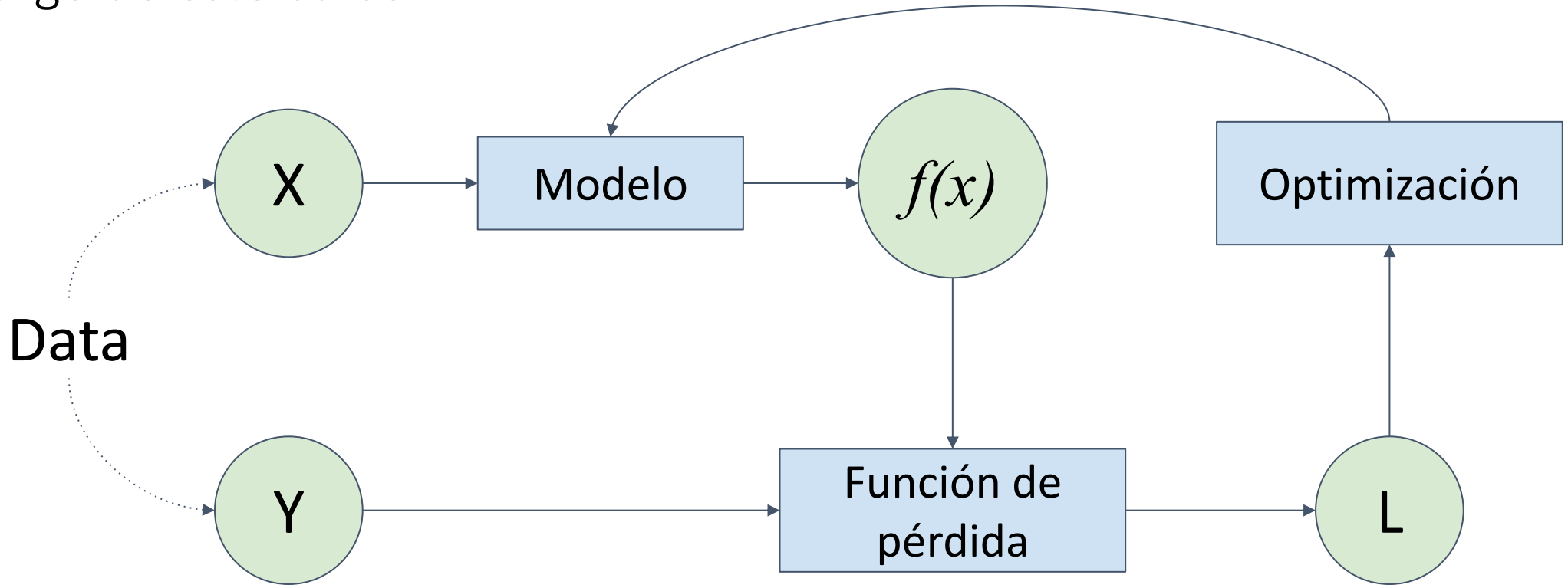
Regresión Lineal

Primer modelo que vamos a aprender.

- **Tarea:** predecir objetivos de valor escalar (ej. precios de acciones)
- **Arquitectura:** función lineal de las entradas

Generalización del Problema

La regresión lineal ejemplifica un enfoque modular que se usará a lo largo de este curso:

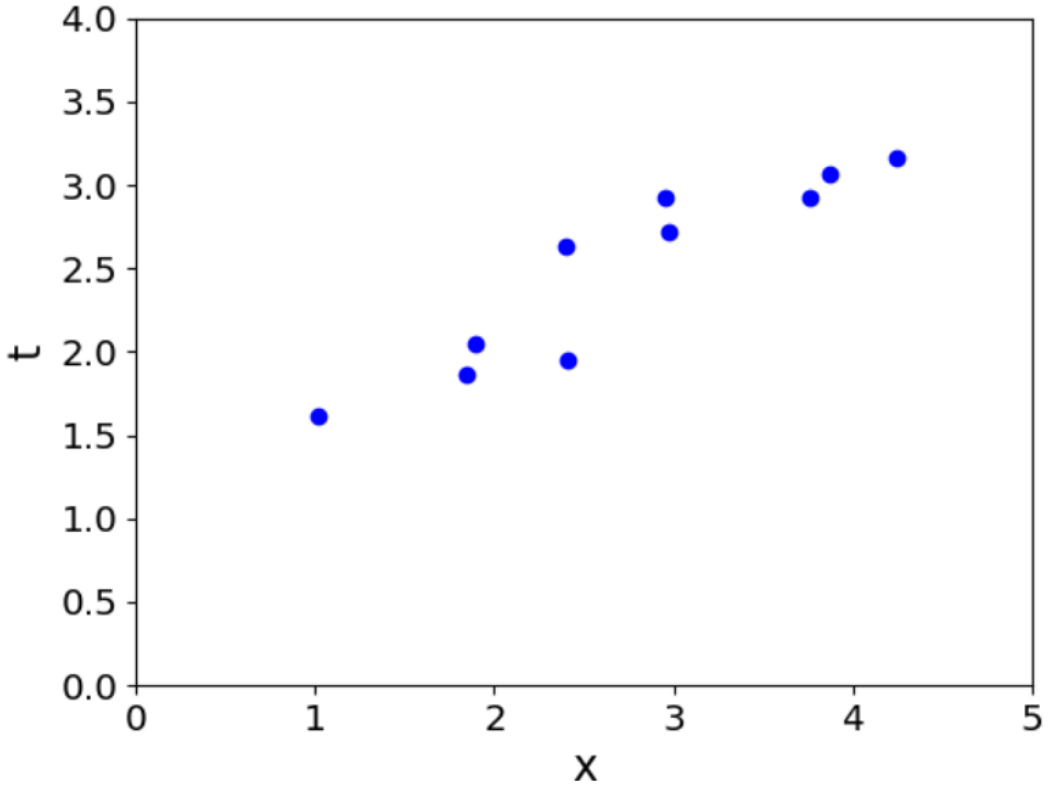


Mezclar y combinar estos componentes modulares nos da muchos métodos nuevos de ML.

Definición matemática formal

En el aprendizaje supervisado:

- Hay una entrada $x \in X$, típicamente un vector de características (o covariables)
- Hay un objetivo (*target*) $t \in T$ (también llamado respuesta, resultado, salida, clase)
- El objetivo es aprender una funciór $f : X \rightarrow T$ tal que $t \approx y = f(x)$ basándose en algunos datos $D = \{(x^{(i)}, t^{(i)}) \text{ para } i = 1, 2, \ldots, N\}$.



Regresión Lineal - Modelo

- **Modelo:** En regresión lineal, usamos una función lineal de las características $\mathbf{x} = (x_1, \ldots, x_D) \in \mathbb{R}^D$ para hacer predicciones y del valor objetivo $t \in \mathbb{R}$:

$$y = f(\mathbf{x}) = \sum_j w_j x_j + b$$

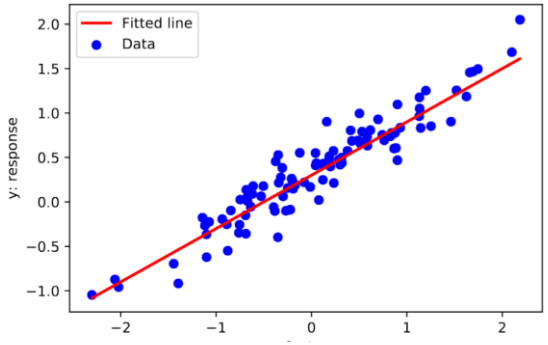
- y es la predicción
- **w** son los pesos
- b es el sesgo (o intercepto)

- **w** y b juntos son los parámetros
- Esperamos que nuestra predicción esté cerca del objetivo: $y \approx t$.

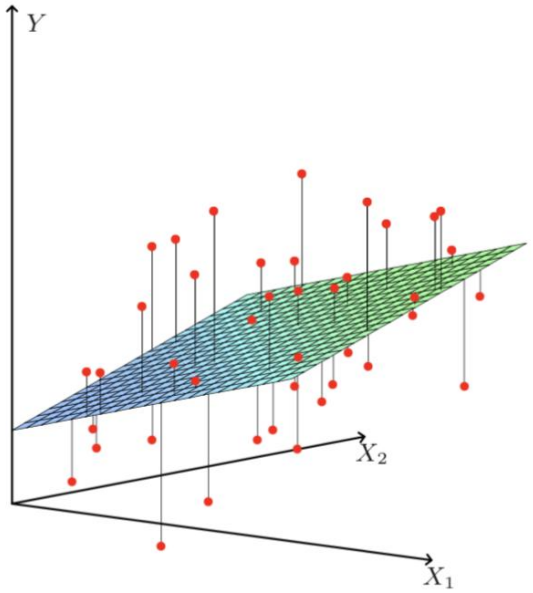
¿Cómo se puede escribir de forma vectorial?

$$y = f(\mathbf{x}) = \sum_j w_j x_j + b$$

¿Qué es Lineal? 1 característica vs D características



- Si tenemos solo 1 característica:
  - $y = wx + b$ donde $w, x, b \in \mathbb{R}$.
- y es lineal en x.



- Si tenemos D características:
  - $y = w^T x + b$ donde $w, x \in \mathbb{R}^D$, $b \in \mathbb{R}$
- y es lineal en x.

Regresión Lineal - Función de Pérdida

- Una función de pérdida (*loss*) L(y, t) define qué tan malo es si, para algún ejemplo x, el algoritmo predice y, pero el objetivo es realmente t.
- Función de pérdida de error cuadrático:

$$\mathcal{L}(y,t) = \frac{1}{2}(y-t)^2$$

- y − t es el residuo, y queremos que sea pequeño en magnitud
- El factor 1/2 es solo para hacer los cálculos convenientes.

Función de costo

- **Función de costo:** función de pérdida promediada sobre todos los ejemplos de entrenamiento

$$\mathcal{J}(\mathbf{w}, b) = \frac{1}{2N} \sum_{i=1}^{N} \left(y^{(i)} - t^{(i)} \right)^2$$

$$= \frac{1}{2N} \sum_{i=1}^{N} \left(\mathbf{w}^\top \mathbf{x}^{(i)} + b - t^{(i)} \right)^2$$

- La terminología varía. Algunos llaman al "costo" pérdida empírica o promedio.



Vectorización

- La predicción para un punto de datos puede calcularse usando un bucle for:

```
y = b
for j in range(M):
    y += w[j] * x[j]
```

- Los super/subíndices excesivos son difíciles de manejar, y los bucles de Python son lentos, así que vectorizamos los algoritmos expresándolos en términos de vectores y matrices.

$$\mathbf{w} = (w_1, \ldots, w_D)^\top \qquad \mathbf{x} = (x_1, \ldots, x_D)^\top$$

$$y = \mathbf{w}^\top \mathbf{x} + b$$

```
y = np.dot(w, x) + b
```

¿Para qué vectorizamos?



¿Por qué vectorizar?

- Las ecuaciones, y el código, serán más simples y legibles. ¡Elimina variables/índices auxiliares!
- El código vectorizado es mucho más rápido
  - Reduce la sobrecarga del intérprete de Python
  - Usa bibliotecas de álgebra lineal altamente optimizadas (soporte de hardware)
  - La multiplicación de matrices es muy rápida en GPUs (Unidad de Procesamiento Gráfico)

Cambiar entre forma vectorizada y no vectorizada es una habilidad que se gana con la práctica

- Algunas derivaciones son más fáciles de hacer elemento por elemento
- Algunos algoritmos son más fáciles de escribir/entender usando bucles for y vectorizar después para rendimiento

Vectorización

- Podemos organizar todos los ejemplos de entrenamiento en una **matriz de diseño** **X** con una fila por ejemplo de entrenamiento, y todos los objetivos en el **vector objetivo** **t**.

one feature across all training examples

$$\mathbf{X} = \begin{pmatrix} \mathbf{x}^{(1)\top} \\ \mathbf{x}^{(2)\top} \\ \mathbf{x}^{(3)\top} \end{pmatrix} = \begin{pmatrix} 8 & 0 & 3 & 0 \\ 6 & -1 & 5 & 3 \\ 2 & 5 & -2 & 8 \end{pmatrix}$$

one training example (vector)

- Calculando las predicciones para todo el conjunto de datos:

$$\mathbf{X}\mathbf{w} + b\mathbf{1} = \begin{pmatrix} \mathbf{w}^T\mathbf{x}^{(1)} + b \\ \vdots \\ \mathbf{w}^T\mathbf{x}^{(N)} + b \end{pmatrix} = \begin{pmatrix} y^{(1)} \\ \vdots \\ y^{(N)} \end{pmatrix} = \mathbf{y}$$



Vectorización

- Calculando el costo del error cuadrático en todo el conjunto de datos:

$$\mathbf{y} = \mathbf{X}\mathbf{w} + b\mathbf{1}$$

$$\mathcal{J} = \frac{1}{2N}\|\mathbf{y} - \mathbf{t}\|^2$$

- A veces podemos usar J = ½||y − t||², sin normalizador. Esto correspondería a la suma de pérdidas, y no a la pérdida promediada. El minimizador no depende de N (¡pero la optimización podría!).
- También podemos agregar una columna de 1s a la matriz de diseño, combinar el sesgo y los pesos, y escribir convenientemente

$$\mathbf{X} = \begin{bmatrix} 1 & [\mathbf{x}^{(1)}]^\top \\ 1 & [\mathbf{x}^{(2)}]^\top \\ 1 & \vdots \end{bmatrix} \in \mathbb{R}^{N\times(D+1)} \quad \text{and } \mathbf{w} = \begin{bmatrix} b \\ w_1 \\ w_2 \\ \vdots \end{bmatrix} \in \mathbb{R}^{D+1}$$

Entonces, nuestras predicciones se reducen a **y = Xw**.

Resolviendo el Problema de Minimización

Definimos una función de costo J(w). Esto es lo que queremos minimizar.

Recordemos del cálculo: el mínimo de una función suave (si existe) ocurre en un punto crítico, es decir, un punto donde la derivada es cero.

- Generalización multivariada: establecer las derivadas parciales $\partial J/\partial w_j$ a cero.
- Equivalentemente, podemos establecer el gradiente a cero. El gradiente es el vector de derivadas parciales:

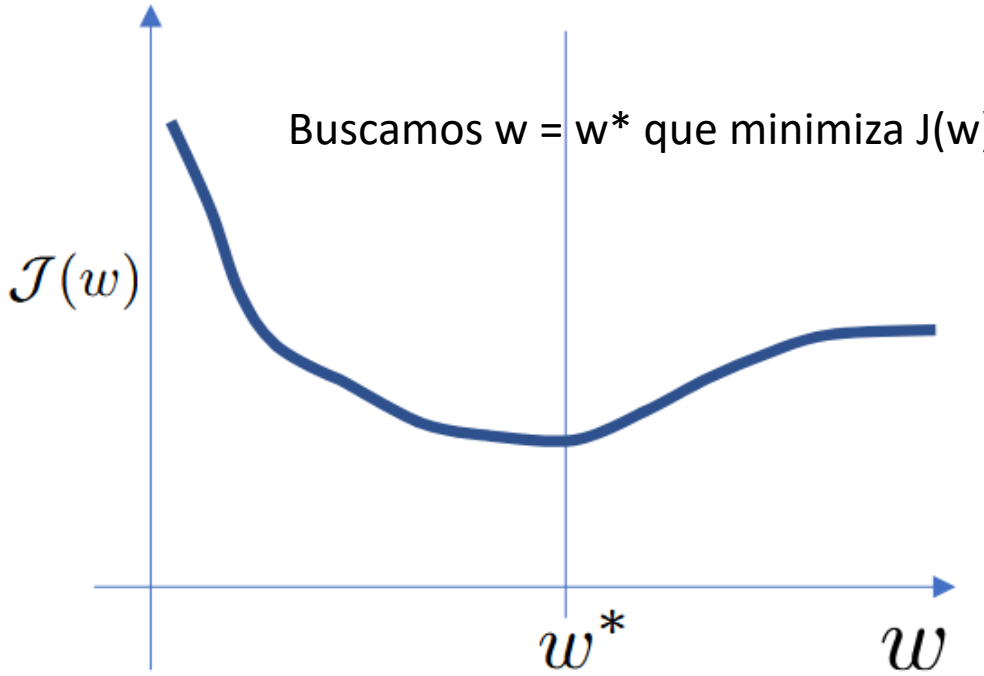
$$\nabla_{\mathbf{w}} \mathcal{J} = \frac{\partial \mathcal{J}}{\partial \mathbf{w}} = \begin{pmatrix} \frac{\partial \mathcal{J}}{\partial w_1} \\ \vdots \\ \frac{\partial \mathcal{J}}{\partial w_D} \end{pmatrix}$$

Las soluciones pueden ser directas o iterativas

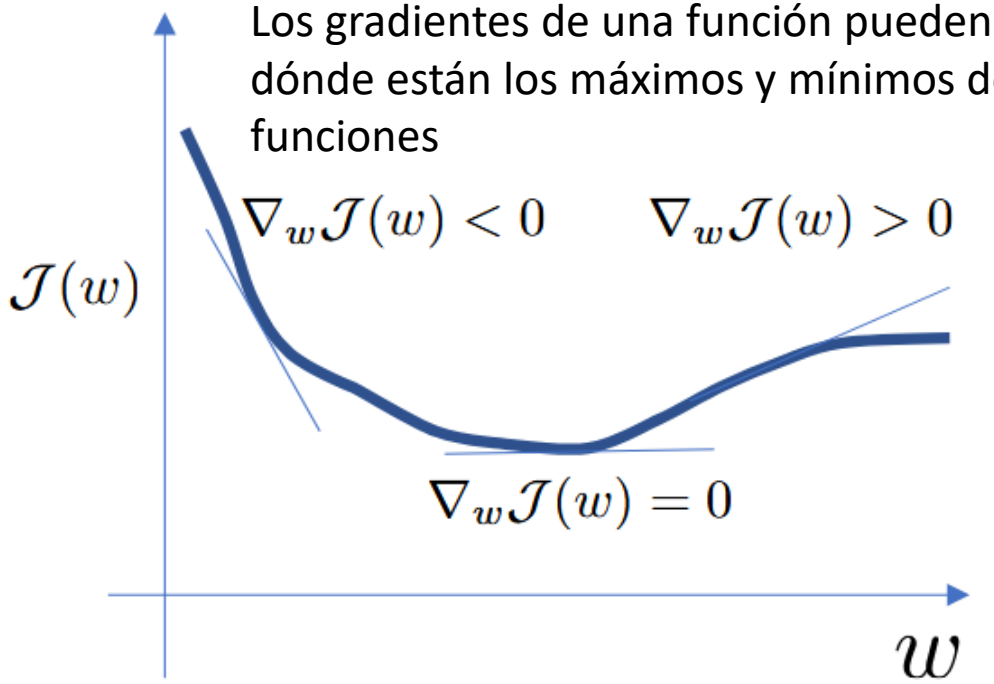
- A veces podemos encontrar directamente parámetros probablemente óptimos (ej. establecer el gradiente a cero y resolver en forma cerrada). A esto lo llamamos solución directa.
- Los métodos de solución iterativa aplican repetidamente una regla de actualización que gradualmente nos acerca a la solución.

Solución Directa: Cálculo

- Consideremos una visualización de J(w) donde w es unidimensional



Los gradientes de una función pueden decirnos dónde están los máximos y mínimos de las funciones



Estrategia: Escribir una expresión algebraica para $\nabla_\varphi J(w)$. Establecer la ecuación a 0. Resolver para w

Solución Directa: Cálculo

- Buscamos w para minimizar $J(w) = \frac{1}{2}||Xw - t||^2$
- Tomando el gradiente con respecto a w e igualándolo a 0, obtenemos:

$$\underset{\mathbf{w}}{\text{minimize}}\, \mathcal{J}(\mathbf{w}) =: \frac{1}{2}\|\mathbf{t} - \mathbf{X}\mathbf{w}\|_2^2.$$

$$\mathcal{J}(\mathbf{w}) = \frac{1}{2}\|\mathbf{t}\|_2^2 + \frac{1}{2}\mathbf{w}^\top \mathbf{X}^\top \mathbf{X}\mathbf{w} - \mathbf{t}^\top \mathbf{X}\mathbf{w}.$$

$$\nabla \mathcal{J}(\mathbf{w}) = \mathbf{X}^\top \mathbf{X}\mathbf{w} - \mathbf{X}^\top \mathbf{t} = 0.$$



Solución Directa: Cálculo

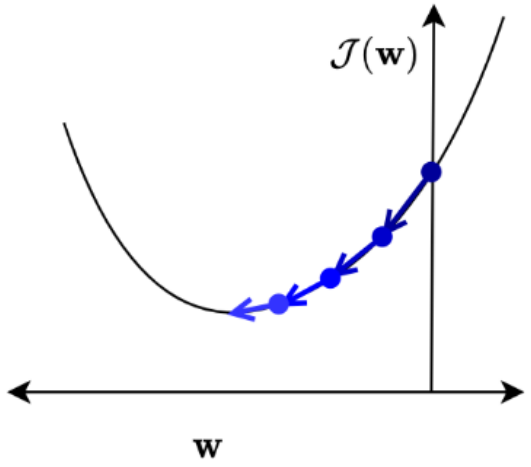
- Pesos óptimos:

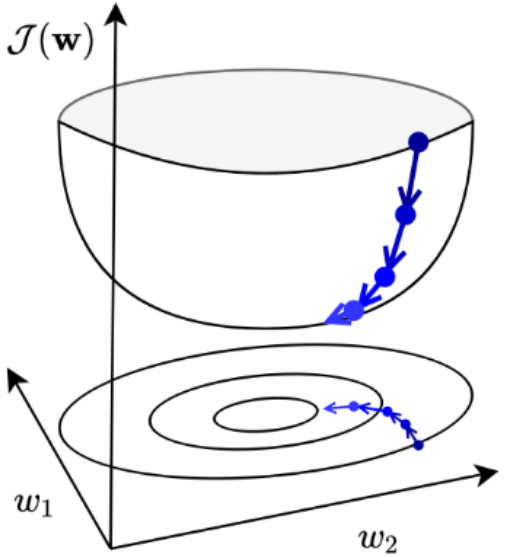
$$\mathbf{w}^* = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{t}$$

- La regresión lineal es uno de solo un puñado de modelos en este curso que permiten solución directa.

Solución Iterativa: Descenso de Gradiente

- La mayoría de los problemas de optimización que cubrimos en este curso no tienen solución directa.
- Ahora veamos una segunda forma de minimizar la función de costo que es más ampliamente aplicable: descenso de gradiente.
- El descenso de gradiente es un algoritmo iterativo, lo que significa que aplicamos una actualización repetidamente hasta que se cumple algún criterio.
- Inicializamos los pesos a algo razonable (ej. todos ceros) y los ajustamos repetidamente en la dirección de descenso más pronunciado.





Descenso de Gradiente

- Observemos:
- si $\partial J/\partial w_j > 0$, entonces aumentar w_j aumenta J.
- si $\partial J/\partial w_j < 0$, entonces aumentar w_j disminuye J.
- La siguiente actualización siempre disminuye la función de costo para α suficientemente pequeño (a menos que $\partial J/\partial w_j = 0$):

$$w_j \leftarrow w_j - \alpha \frac{\partial \mathcal{J}}{\partial w_j}$$

- $\alpha > 0$ es una tasa de aprendizaje (o tamaño de paso). Cuanto más grande sea, más rápido cambia **w**. Veremos más adelante cómo ajustar la tasa de aprendizaje, pero los valores son típicamente pequeños, ej. 0.01 o 0.0001.
- Si el costo es la suma de N *losses* individuales en lugar de su promedio, se necesitará una tasa de aprendizaje más pequeña ($\alpha' = \alpha/N$).

Descenso de Gradiente

- Esto obtiene su nombre del gradiente. Recordemos la definición:
  - Esta es la dirección de aumento más rápido en J.

$$\nabla_{\mathbf{w}} \mathcal{J} = \frac{\partial \mathcal{J}}{\partial \mathbf{w}} = \begin{pmatrix} \frac{\partial \mathcal{J}}{\partial w_1} \\ \vdots \\ \frac{\partial \mathcal{J}}{\partial w_D} \end{pmatrix}$$

- Regla de actualización en forma vectorial:

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha \frac{\partial \mathcal{J}}{\partial \mathbf{w}}$$

- Y para regresión lineal tenemos:

$$\mathbf{w} \leftarrow \mathbf{w} - \frac{\alpha}{N} \sum_{i=1}^{N} (y^{(i)} - t^{(i)}) \, \mathbf{x}^{(i)}$$

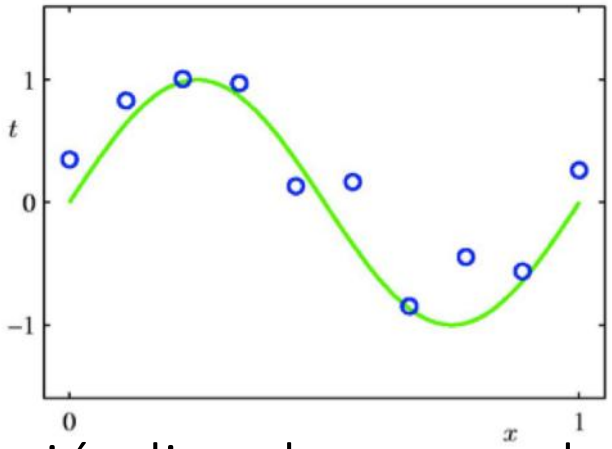


Descenso de Gradiente para Regresión Lineal

- Incluso para regresión lineal, donde hay solución directa, a veces necesitamos usar DG.
- ¿Por qué descenso de gradiente, si podemos encontrar el óptimo directamente?
  - DG puede aplicarse a un conjunto mucho más amplio de modelos
  - DG puede ser más fácil de implementar que las soluciones directas
  - Para regresión en espacio de alta dimensión, DG es más eficiente que la solución directa
    - Solución de regresión lineal: $(X^TX)^{-1}X^Tt$
    - La inversión de matrices es un algoritmo $O(D^3)$
    - Cada actualización de DG cuesta $O(ND)$
    - O menos con descenso de gradiente estocástico (SGD, cubierto la próxima semana)
    - Gran diferencia si $D \gg 1$

Mapeo de Características (Expansión de Base)

La relación entre la entrada y la salida puede no ser lineal.



- Aún podemos usar regresión lineal mapeando las características de entrada a otro espacio usando mapeo de características (o expansión de base).
- $\psi(x) : \mathbb{R}^D \rightarrow \mathbb{R}^d$ y tratar la característica mapeada (en $\mathbb{R}^d$) como la entrada de un procedimiento de regresión lineal.Veamos cómo funciona cuando $x \in \mathbb{R}$ y usamos un mapeo de características polinomial.

Mapeo de Características Polinomial

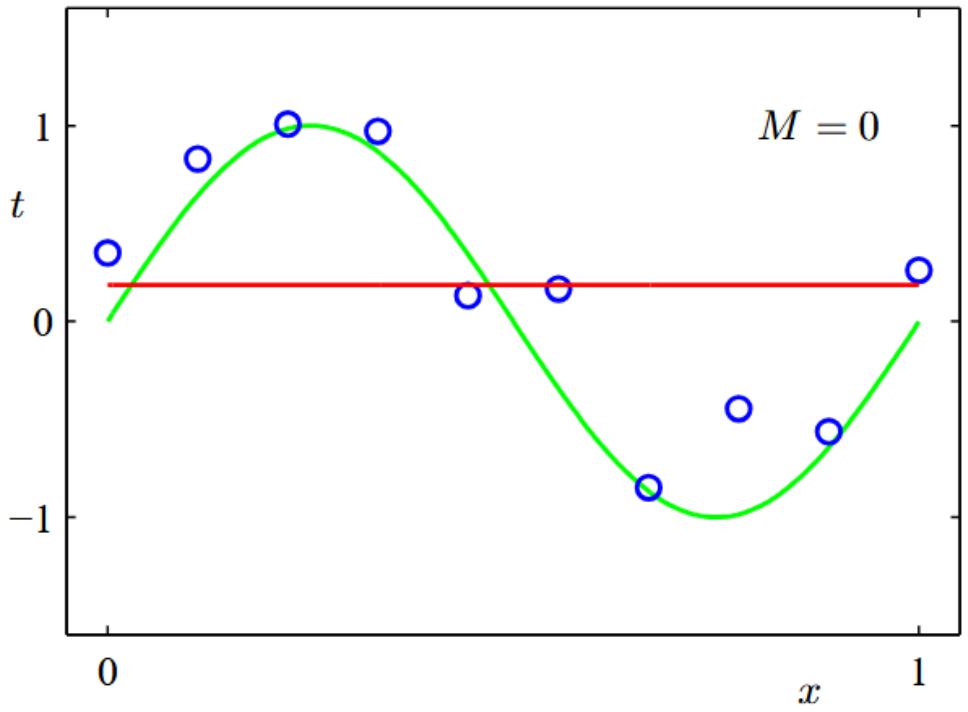
- Si la relación no parece lineal, podemos ajustar un polinomio.
- Ajustar los datos usando una función polinomial de grado M de la forma:

$$y = w_0 + w_1 x + w_2 x^2 + \ldots + w_M x^M = \sum_{i=0}^{M} w_i x^i$$

- Aquí el mapeo de características es $\psi(x) = [1, x, x^2, \ldots, x^M]^T$.
- Aún podemos usar regresión lineal para encontrar w ya que $y = \psi(x)^T w$ es lineal en w_0, w_1, ...
- En general, ψ puede ser cualquier función. Otro ejemplo:$\psi(x) = [1, \sin(2\pi x), \cos(2\pi x), \sin(4\pi x), \ldots]^T$.

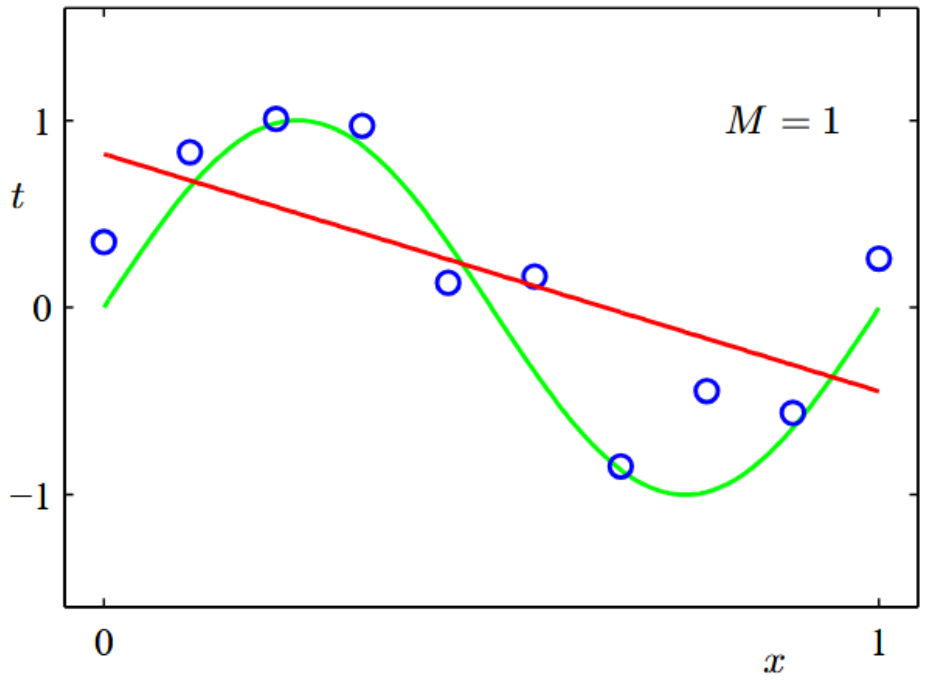
Mapeo Polinomial con M = 0

$$y = w_0$$



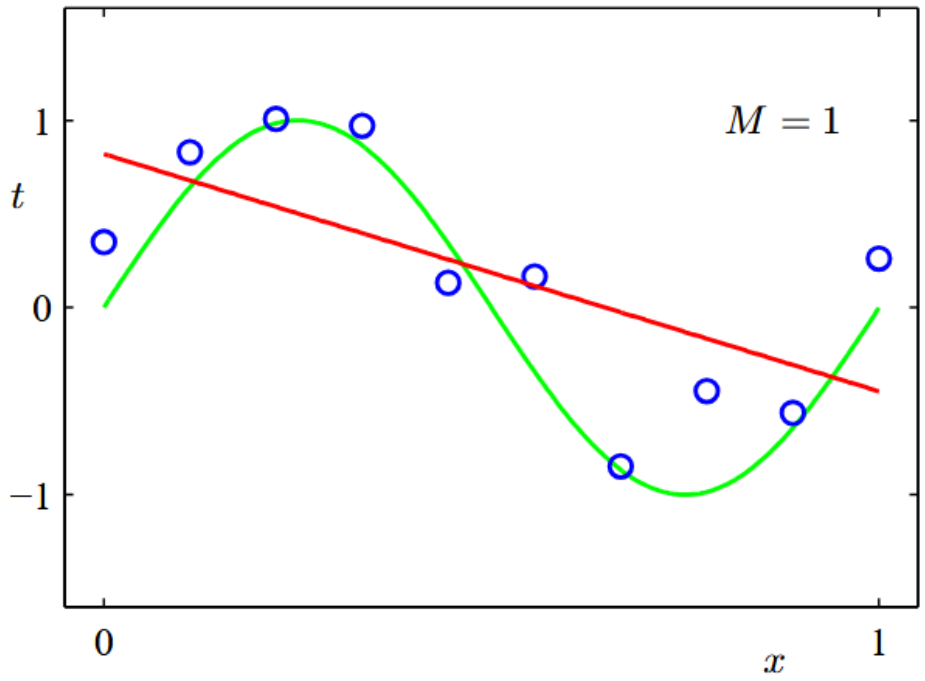
Mapeo Polinomial con M = 1

$$y = w_0 + w_1 x$$



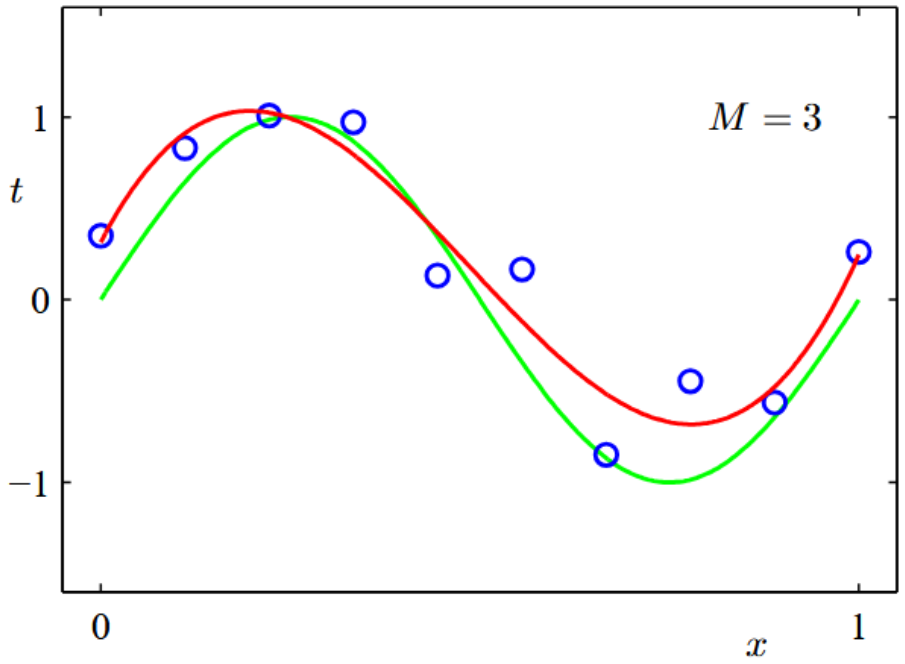
Mapeo Polinomial con M = 2

$$y = w_0 + w_1 x$$



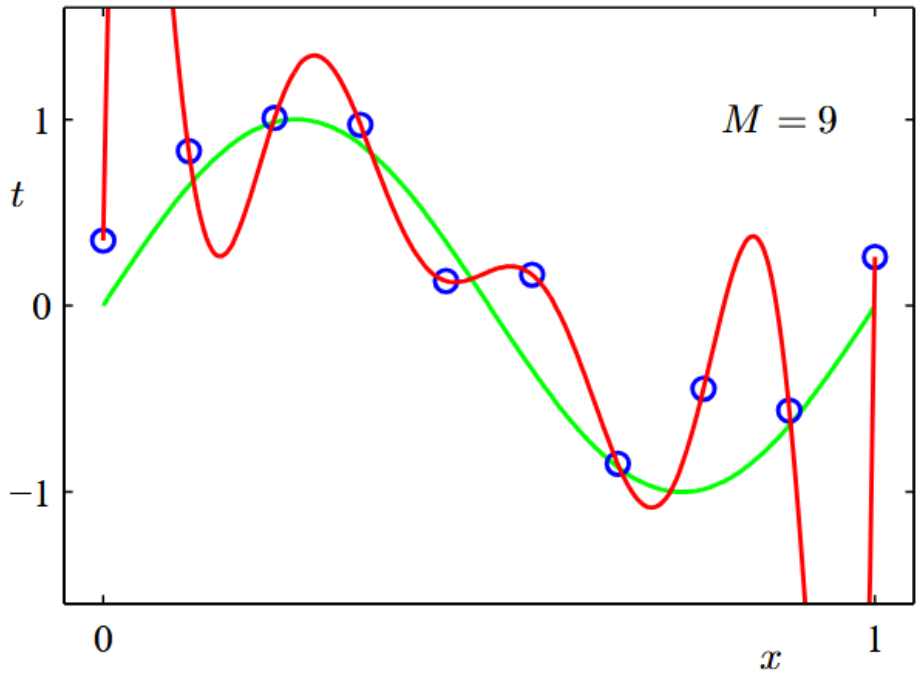
Mapeo Polinomial con M = 3

$$y = w_0 + w_1 x + w_2 x^2 + w_3 x^3$$



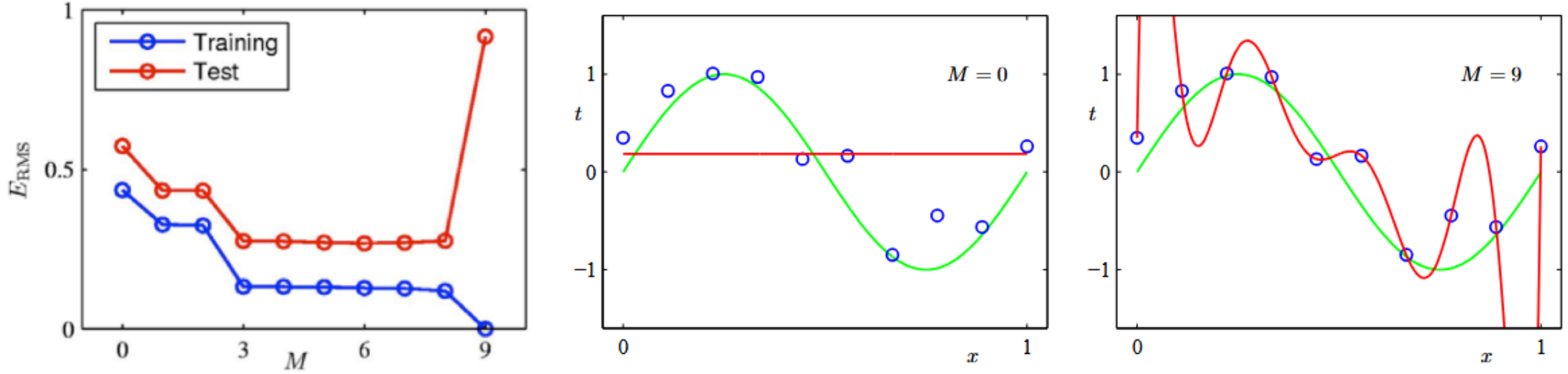
Mapeo Polinomial con M = 9

$$y = w_0 + w_1 x + w_2 x^2 + w_3 x^3 + \ldots + w_9 x^9$$



Complejidad del Modelo y Generalización

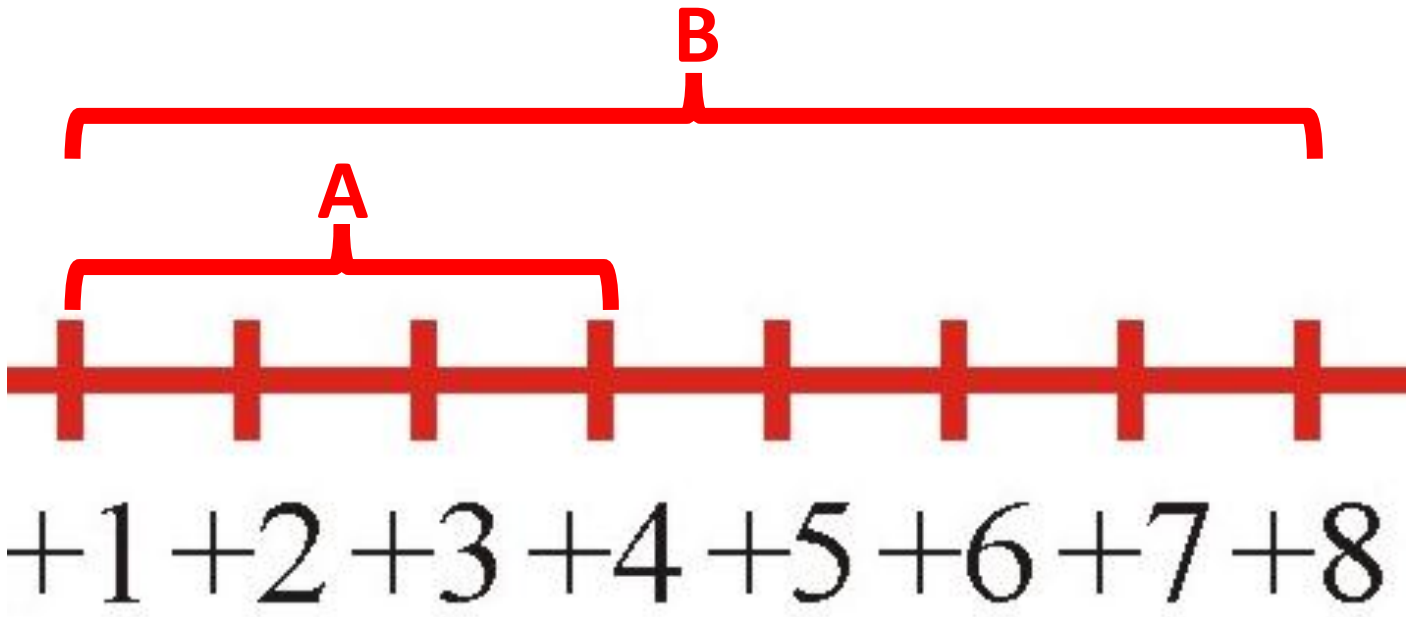
- Subajuste (M=0): el modelo es demasiado simple — no ajusta los datos.
- Sobreajuste (M=9): el modelo es demasiado complejo — ajusta perfectamente.



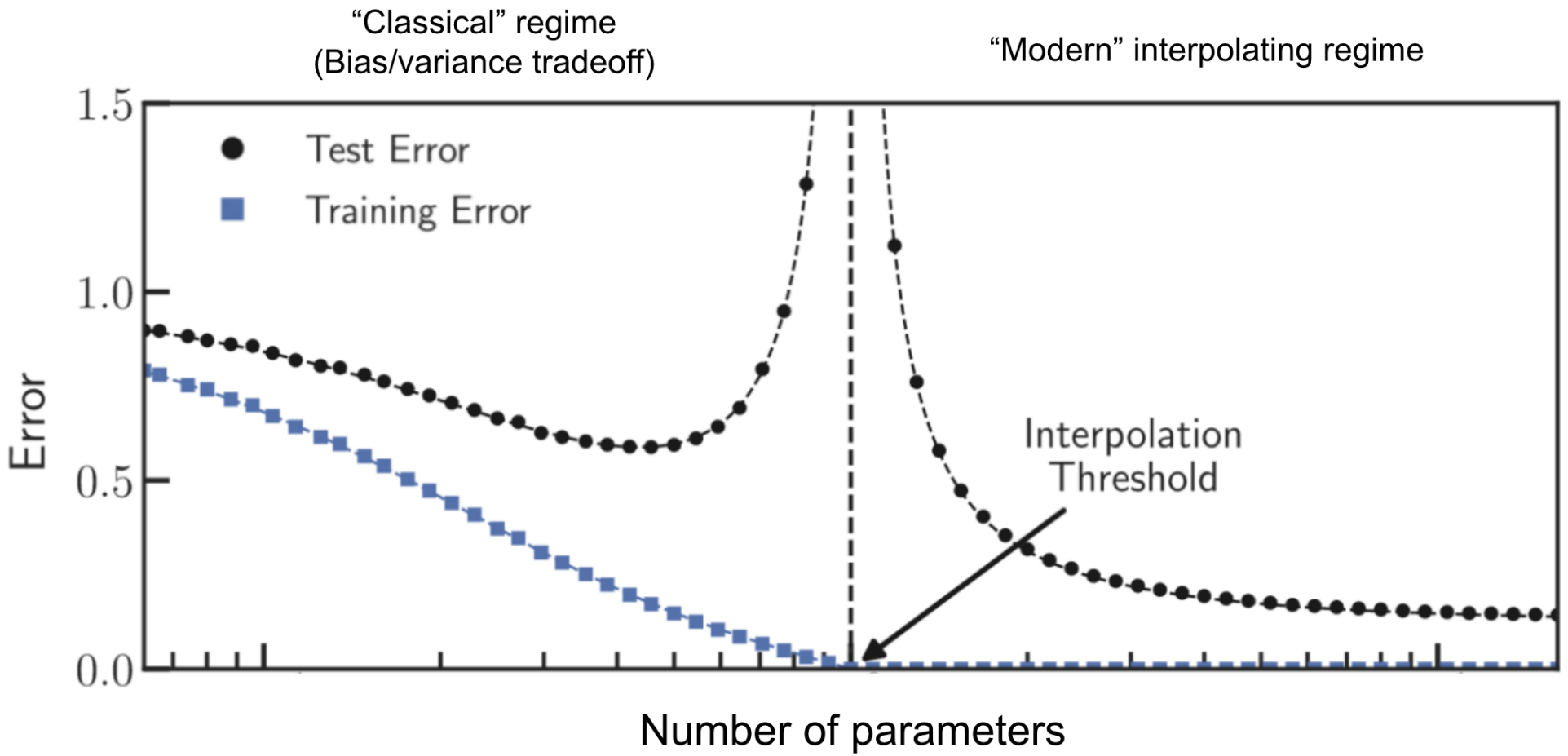
- Buen modelo (M=3): Logra un error de prueba pequeño (generaliza bien).

¿Caso cerrado?

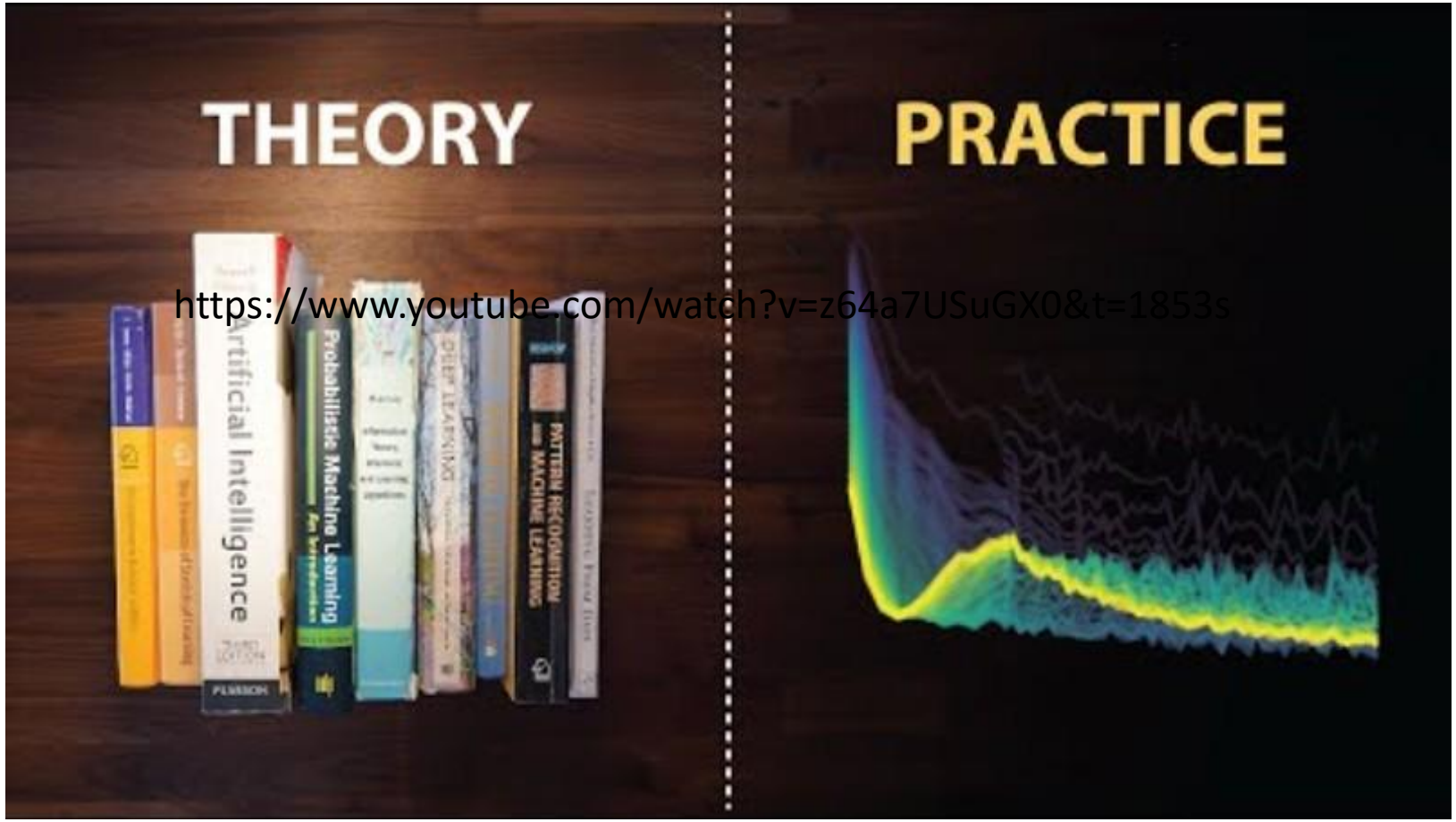
¿Dónde hay más números A o B?



Curva de doble descenso



Doble descenso en polinomios



https://www.youtube.com/watch?v=z64a7USuGX0&t=1853s



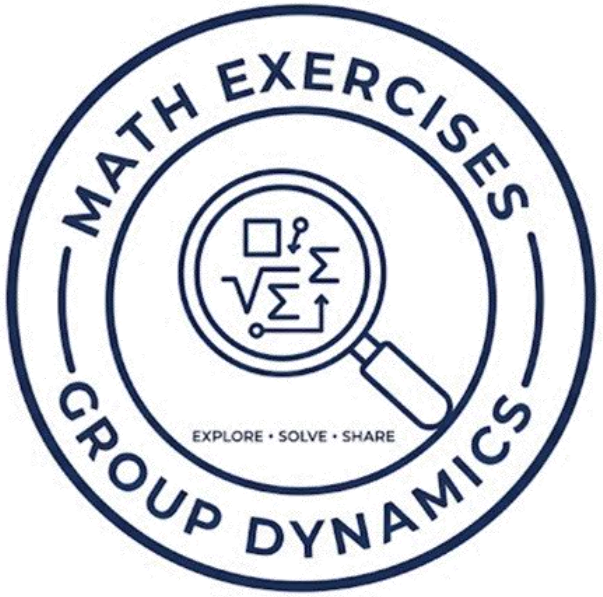
Regularización

- El grado M del polinomio controla la complejidad del modelo.
- El valor de M es un hiperparámetro para la expansión polinomial, igual que k en KNN.
- Podemos ajustarlo usando un conjunto de validación.
- Restringir el número de parámetros / funciones base (M) es un enfoque crudo para controlar la complejidad del modelo.
- Otro enfoque: mantener el modelo grande, pero regularizarlo
- Regularizador: una función que modifica el paisaje de optimización

(10 min) Break



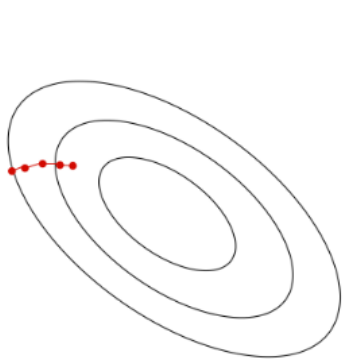
Dinámica Grupal: Encontrar los parámetros



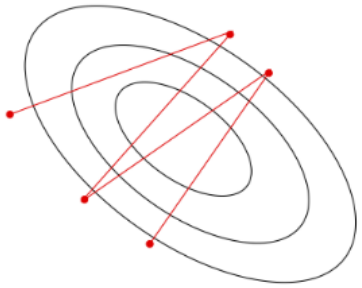
1. Los grupos van participando en orden numérico.
2. **Sugieren** a donde mover los parámetros del modelo.
3. Una vez se repitan los grupos sin mejorar la gradiente se termina la dinámica y el grupo con menor gradiente gana.

Tasa de Aprendizaje (Tamaño del Paso)

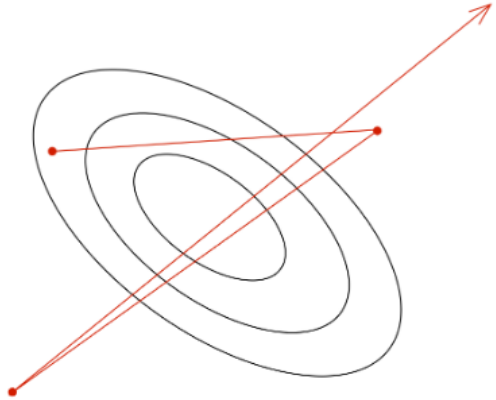
- En el descenso de gradiente, la tasa de aprendizaje α es un hiperparámetro que necesitamos ajustar. Cosas que pueden salir mal:



α too small:
slow progress



α too large:
oscillations

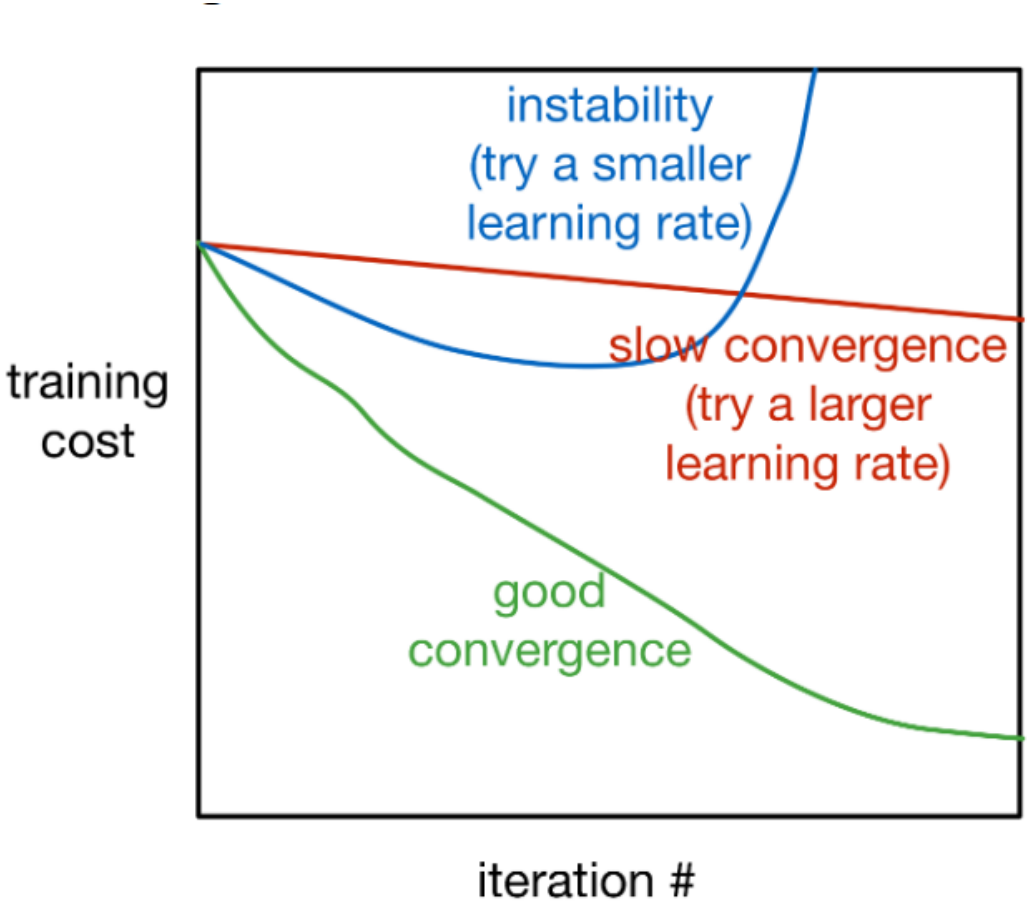


α much too large:
instability

- Los buenos valores están típicamente entre 0.001 y 0.1. Debes hacer una búsqueda en cuadrícula (probar 0.1, 0.03, 0.01, ...).

Curvas de Entrenamiento

- Para diagnosticar problemas de optimización, es útil observar las curvas de entrenamiento: graficar el costo de entrenamiento en función de la iteración.
- Advertencia: es muy difícil determinar a partir de las curvas si un optimizador ha convergido. Pueden revelar problemas importantes, pero no garantizan convergencia.



¿Cual es el problema de usar el descenso de gradiente?

¿Qué es el descenso de gradiente estocástico?

Descenso de Gradiente Estocástico

- La función de costo J ha sido el promedio de la pérdida sobre los ejemplos de entrenamie

$$\mathcal{J}(\boldsymbol{\theta}) = \frac{1}{N}\sum_{i=1}^{N} \mathcal{L}^{(i)} = \frac{1}{N}\sum_{i=1}^{N} \mathcal{L}(y(\mathbf{x}^{(i)}, \boldsymbol{\theta}), t^{(i)}).$$

- (θ denota los parámetros; en regresión lineal, θ = (w, b))
- Calcular el gradiente requiere sumar sobre todos los ejemplos. Esto se conoce como entrenamiento por lotes.
- ¡El entrenamiento por lotes es impráctico si tienes un conjunto de datos grande N ≫ 1!

Descenso de Gradiente Estocástico (SGD)

- SGD actualiza los parámetros basándose en el gradiente de un solo ejemplo:
  - 1. Elegir i uniformemente al azar
  - 2. $\theta \leftarrow \theta - \alpha(\partial L^{(i)}/\partial\theta)$

- ¡El costo de cada actualización SGD es independiente de N!
- ¡SGD puede hacer progreso significativo antes de ver todos los datos!
- Justificación: el gradiente estocástico es un estimador insesgado del gradiente por lotes.

$$\mathbb{E}\left[\frac{\partial \mathcal{L}^{(i)}}{\partial \boldsymbol{\theta}}\right] = \frac{1}{N}\sum_{i=1}^{N}\frac{\partial \mathcal{L}^{(i)}}{\partial \boldsymbol{\theta}} = \frac{\partial \mathcal{J}}{\partial \boldsymbol{\theta}}.$$



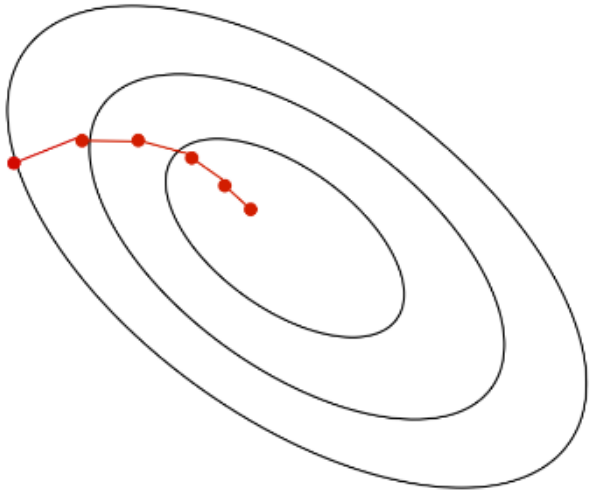
SGD - Problemas y Soluciones

- Problemas con usar un solo ejemplo de entrenamiento:
  - La varianza en la estimación puede ser alta
  - No podemos explotar operaciones vectorizadas eficientes

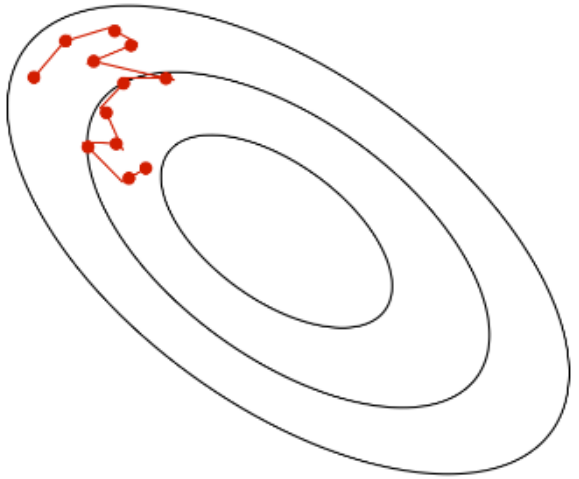
- Enfoque de compromiso: Lotes
  - Calcular gradientes en un conjunto de tamaño medio elegido aleatoriamente
  - Cada pasada sobre los datos se llama una época

SGD vs Descenso de Gradiente por Lotes

- El descenso de gradiente por lotes se mueve directamente cuesta abajo (localmente hablando).
- SGD da pasos en una dirección ruidosa, pero se mueve cuesta abajo en promedio.



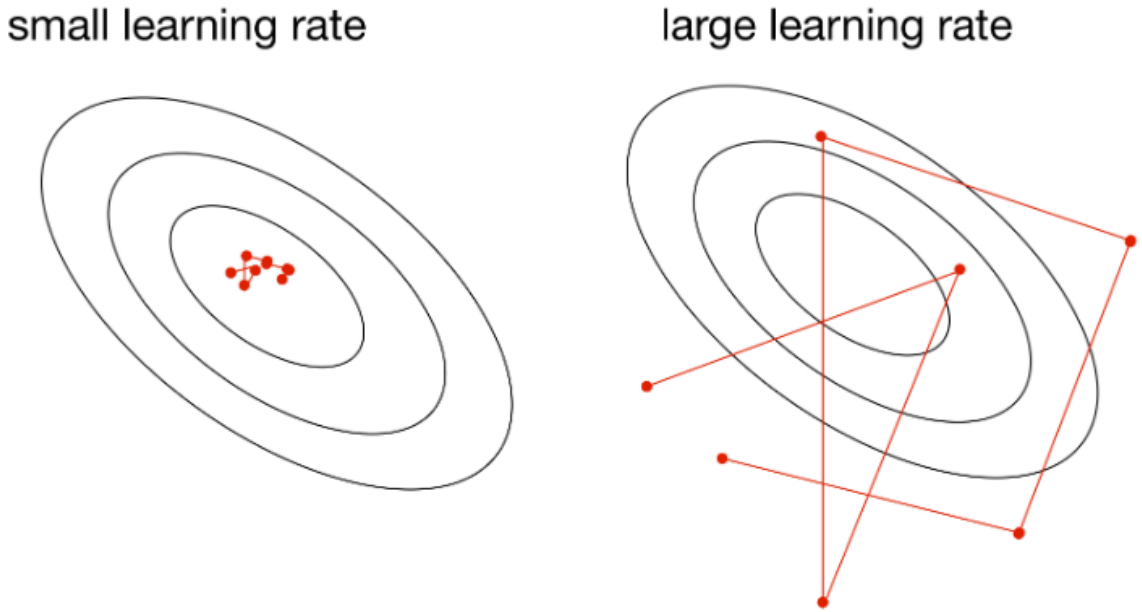
batch gradient descent



stochastic gradient descent

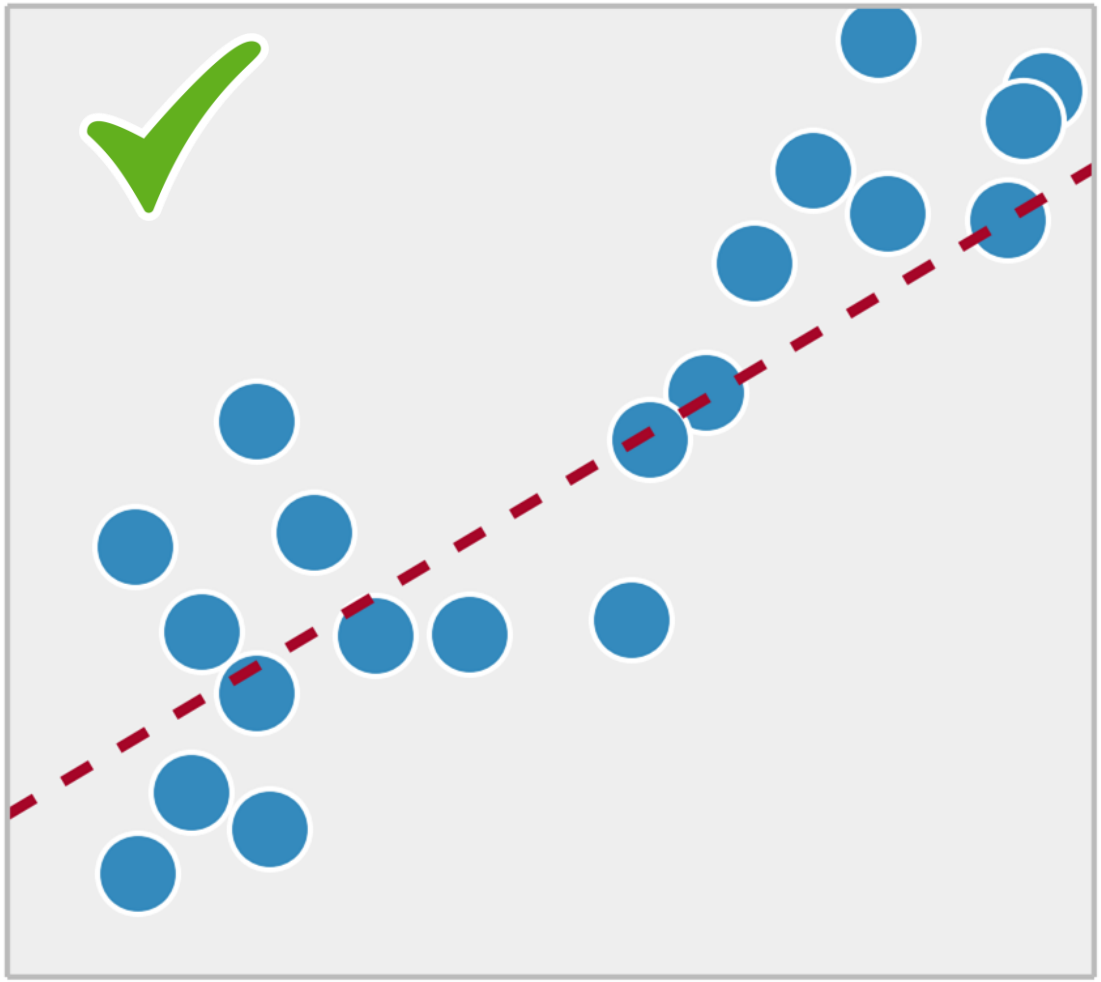
Tasa de Aprendizaje en SGD

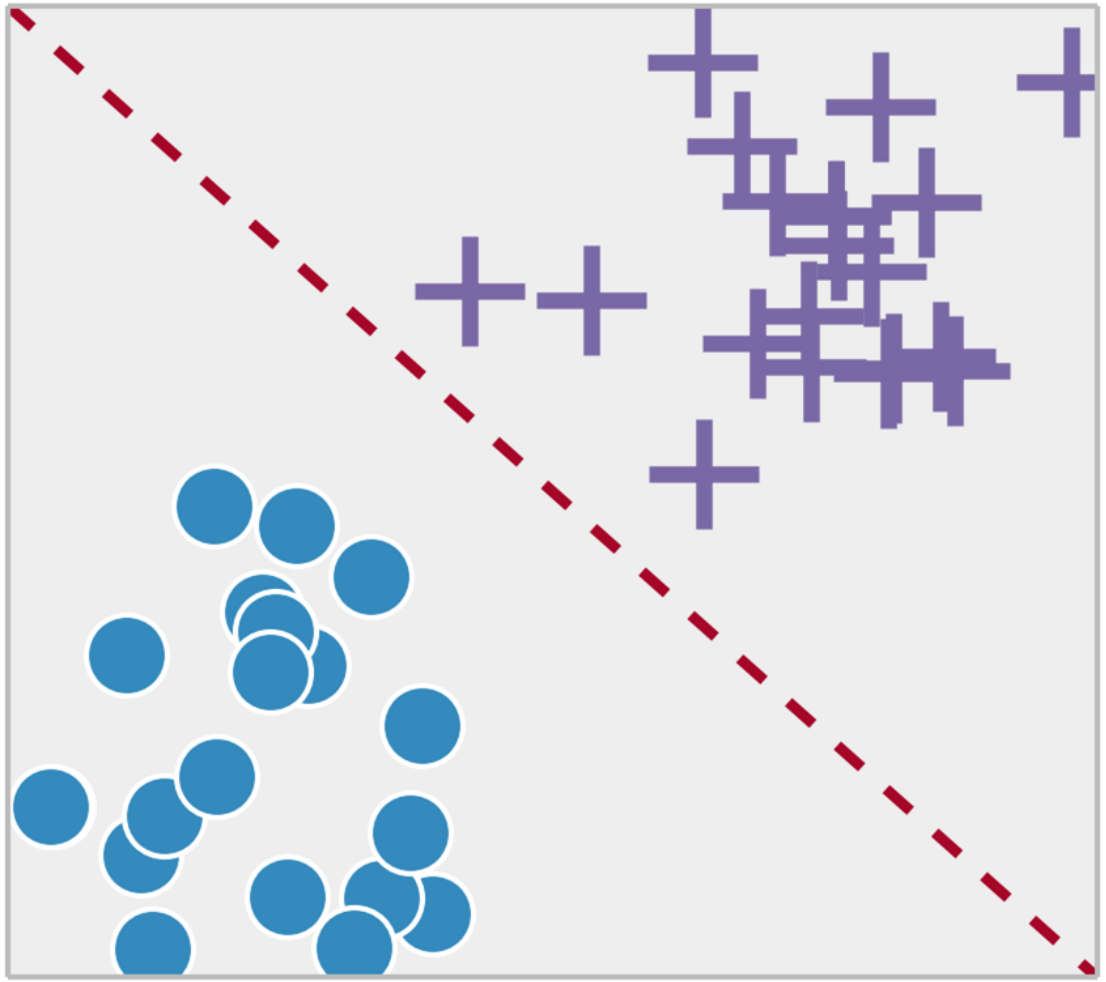
- En el entrenamiento estocástico, la tasa de aprendizaje también influye en la cantidad de ruido en los parámetros.



- Estrategia típica:
  - Usar una tasa de aprendizaje grande al principio para acercarse al óptimo
  - Reducir gradualmente la tasa de aprendizaje para reducir las fluctuaciones

Regresión vs Clasificación







Clasificación Lineal Binaria

Clasificación Lineal Binaria - Resumen

- Clasificación: dado un input D-dimensional $x \in \mathbb{R}^D$, predecir un objetivo de valor discreto
- Binaria: predecir un objetivo binario $t \in \{0, 1\}$
  - Ejemplos con t = 1 son **positivos**, con t = 0 son **negativos (conveniencia comp.)**
- Lineal: la predicción y es una función lineal de x, seguida de un umbral:

$$z = \mathbf{w}^\top \mathbf{x} + b$$

$$y = \begin{cases} 1 & \text{if } z \geq r \\ 0 & \text{if } z < r \end{cases}$$

Simplificaciones

- Eliminando el umbral:
  - Podemos asumir sin pérdida de generalidad que el umbral r = 0

$$\mathbf{w}^\top \mathbf{x} + b \geq r \quad \Longleftrightarrow \quad \mathbf{w}^\top \mathbf{x} + \underbrace{b - r}_{\triangleq w_0} \geq 0.$$

- Eliminando el sesgo:
  - Agregar una característica ficticia $x_0 = 1$. El peso $w_0 = b$ es equivalente a un sesgo
- Modelo simplificado:

Receive input $\mathbf{x} \in \mathbb{R}^{D+1}$ with $x_0 = 1$:

$$z = \mathbf{w}^\top \mathbf{x}$$

$$y = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{if } z < 0 \end{cases}$$



Ejemplos

- Consideremos algunos ejemplos simples para examinar las propiedades de nuestro modelo
- Enfoquémonos en minimizar el error del conjunto de entrenamiento, y olvidémonos de si nuestro modelo generalizará a un conjunto de prueba.



Ejemplo: NOT (NO)

x_0	x_1	t
1	0	1
1	1	0

- Condiciones para clasificación perfecta:
  - Cuando $x_1 = 0$: necesitamos $z = w_0 x_0 + w_1 x_1 \geq 0 \iff w_0 \geq 0$
  - Cuando $x_1 = 1$: necesitamos $z = w_0 x_0 + w_1 x_1 < 0 \iff w_0 + w_1 < 0$

- Solución ejemplo: $w_0 = 1$, $w_1 = -2$

¿Es esta la única solución?



Ejemplo: AND (Y)

x_1	x_2	t
0	0	0
0	1	0
1	0	0
1	1	1

(Puntos para quien encuentra un modelo lineal)

Ejemplo: AND (Y)

x_0	x_1	x_2	t
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1

$$z = w_0 x_0 + w_1 x_1 + w_2 x_2$$

$$\text{need: } w_0 < 0$$

$$\text{need: } w_0 + w_2 < 0$$

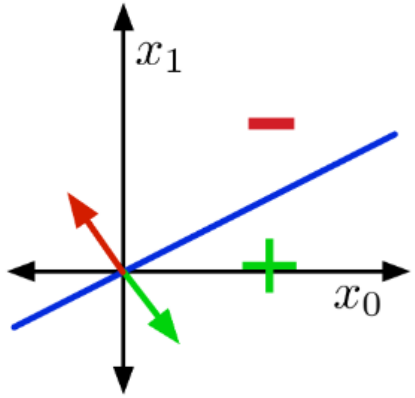
$$\text{need: } w_0 + w_1 < 0$$

$$\text{need: } w_0 + w_1 + w_2 \geq 0$$

- Solución ejemplo: $w_0 = -1.5$, $w_1 = 1$, $w_2 = 1$

La Imagen Geométrica - Espacio de Entrada

Input Space, or Data Space for NOT example

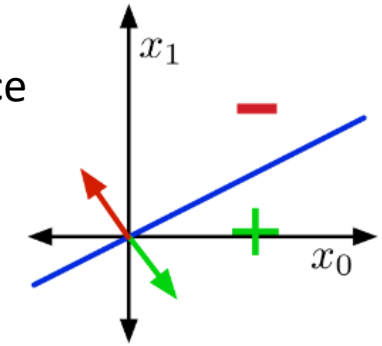


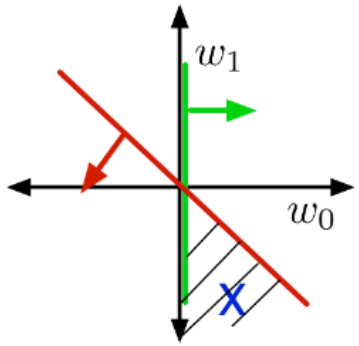
x_0	x_1	t
1	0	1
1	1	0

- Los ejemplos de entrenamiento son puntos
- Los pesos (hipótesis) w pueden representarse mediante semi-espacios
  - $H_+ = \{x : w^T x \geq 0\}$, $H_- = \{x : w^T x < 0\}$
- El límite es la **frontera de decisión**: $\{x : w^T x = 0\}$
  - En 2-D es una línea, en altas dimensiones es un hiperplano
- Si los ejemplos pueden ser perfectamente separados por una regla lineal, **los datos son linealmente separables**.

La Imagen Geométrica - Espacio de Pesos

Weight Space





$$w_0 \geq 0$$

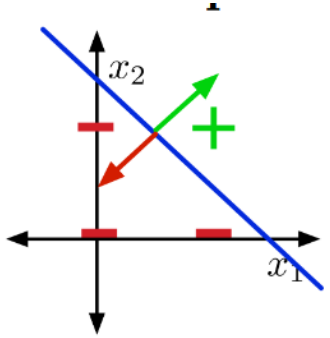
$$w_0 + w_1 < 0$$

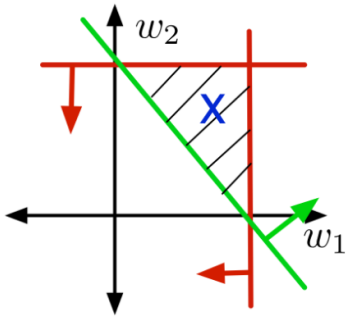
- Los pesos (hipótesis) w son puntos
- Cada ejemplo de entrenamiento x especifica un semi-espacio en el que w debe estar para ser correctamente clasificado
- La región que satisface todas las restricciones es la región factible
  - Si es no vacía, el problema es factible
  - De lo contrario es infactible

La Imagen Geométrica - Ejemplo AND

- El ejemplo **AND (Y)** requiere tres dimensiones, incluyendo la dummy.
- Para visualizar el espacio de datos y el espacio de pesos para un ejemplo 3-D, podemos ver un corte 2-D.
- Las visualizaciones son similares.
  - El conjunto factible siempre tendrá una esquina en el origen.

Visualizaciones del Ejemplo AND





- Espacio de Datos:
  - Corte para $x_0 = 1$
  - Solución: $w_0 = -1.5$, $w_1 = 1$, $w_2 = 1$
  - Frontera de decisión: $-1.5 + x_1 + x_2 = 0$

- Espacio de Pesos:
  - Corte para $w_0 = -1.5$ mostrando las restricciones
    - $w_0 < 0$
    - $w_0 + w_2 < 0$
    - $w_0 + w_1 < 0$
    - $w_0 + w_1 + w_2 \geq 0$

Resumen - Clasificadores Lineales Binarios

- **Resumen:** Objetivos $t \in \{0, 1\}$, inputs $x \in \mathbb{R}^{D+1}$ con $x_0 = 1$

$$z = \mathbf{w}^\top \mathbf{x}$$

$$y = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{if } z < 0 \end{cases}$$

¿Cómo encontrar buenos valores para **w**?

- Si es linealmente separable: programación lineal
- Si no es linealmente separable: el problema es más difícil
  - Los datos casi nunca son linealmente separables en la vida real.

¿Qué podríamos utilizar para llegar a una clasificación con datos linealmente no separables ?

Funciones de Pérdida

- ¿Qué pasa si el conjunto de datos no es linealmente separable?
  - Definir una función de pérdida y minimizar su promedio

- Función aparentemente obvia: $$\mathcal{L}_{0-1}(y,t) = \begin{cases} 0 & \text{if } y = t \\ 1 & \text{if } y \neq t \end{cases} = \mathbb{I}[y \neq t]$$

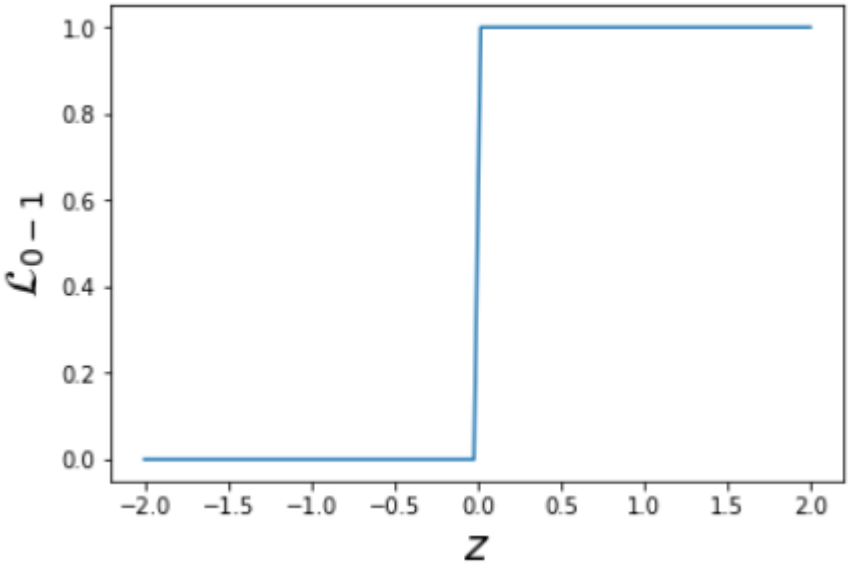
- El costo J es la tasa de clasificación errónea: $$\mathcal{J} = \frac{1}{N}\sum_{i=1}^{N} \mathbb{I}[y^{(i)} \neq t^{(i)}]$$

¿Cúal es el problema con este enfoque?

Intento 1: Pérdida (loss) 0-1

- El mínimo de una función estará en sus puntos críticos.
- Regla de la cadena:

$$\frac{\partial \mathcal{L}_{0-1}}{\partial w_j} = \frac{\partial \mathcal{L}_{0-1}}{\partial z} \frac{\partial z}{\partial w_j}$$



- Problema: $\partial L_{0-1}/\partial z$ es cero en todas partes donde está definido!
  - Cambiar los pesos por una cantidad muy pequeña no tiene efecto en la pérdida
  - ¡Casi cualquier punto tiene gradiente 0!

Intento 2: Regresión Lineal

- A veces podemos reemplazar la función de pérdida que nos importa con una más fácil de optimizar.
  - Esto se conoce como relajación con una función de pérdida sustituta suave.

- Un problema con L_{0-1}: definida en términos de la predicción final, que involucra una discontinuidad

- En su lugar, definir la pérdida en términos de $w^T x$ directamente
  - Notación: $z = w^T x$

Intento 2: Regresión Lineal (cont.)

- Ya sabemos cómo ajustar un modelo de regresión lineal. ¿Podemos usarlo?

$$z = \mathbf{w}^\top \mathbf{x}$$

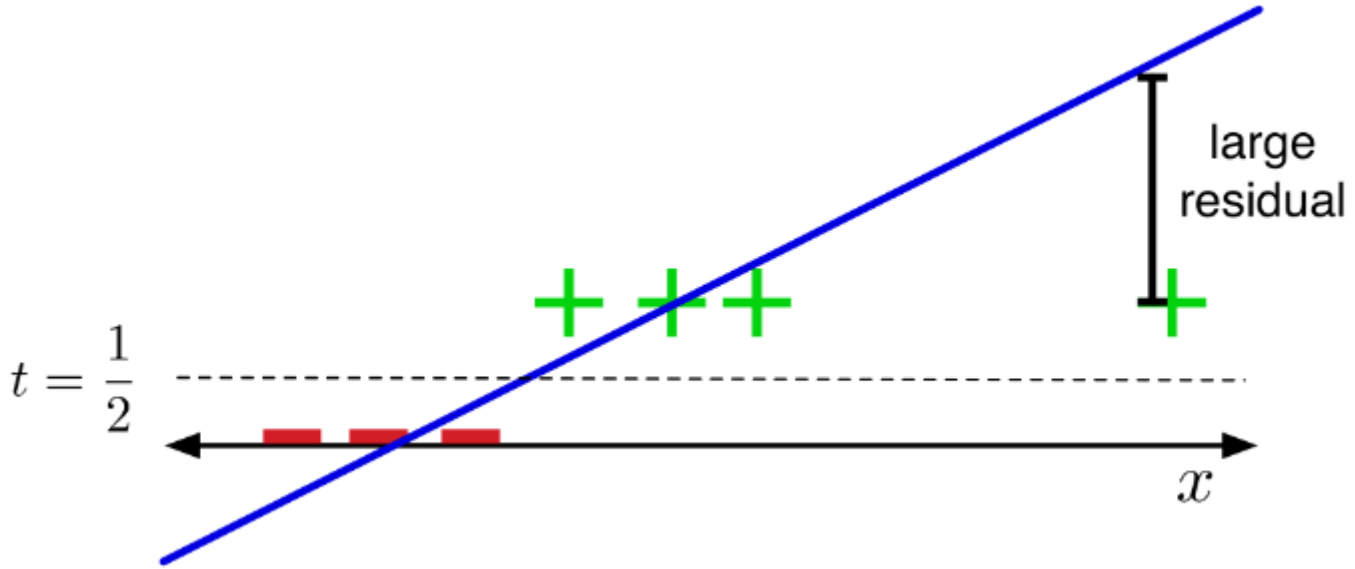
$$\mathcal{L}_{\mathrm{SE}}(z, t) = \frac{1}{2}(z - t)^2$$

- No importa que los objetivos sean binarios. Tratarlos como valores continuos.

- Para esta pérdida, tiene sentido umbralizar z en ½ (explicar porque)

Intento 2: El Problema

El problema:

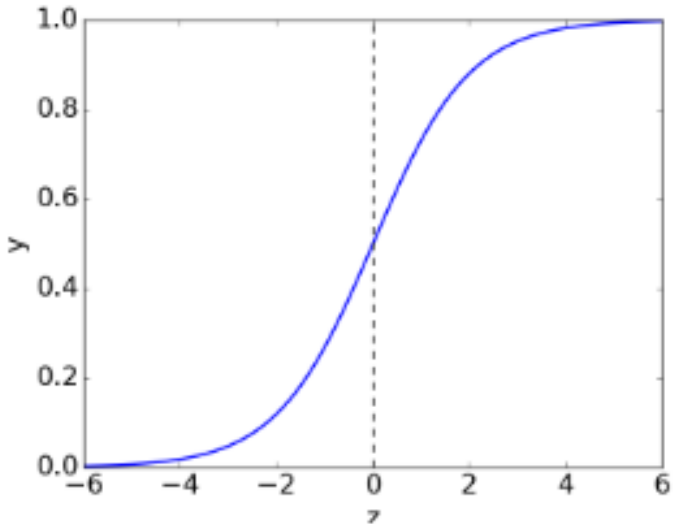


- ¡La función de pérdida odia cuando haces predicciones correctas con alta confianza!
- Si t = 1, está más descontenta con z = 10 que con z = 0.

Intento 3: Función de Activación Logística

- Obviamente no hay razón para predecir valores fuera de [0, 1]. Comprimamos y en este intervalo.
- La función logística es un tipo de sigmoide (función en forma de S):

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$



- $\sigma^{-1}(y) = \log(y/(1-y))$ se llama el **logit**.
- Modelo log-lineal:

$$z = \mathbf{w}^\top \mathbf{x}$$

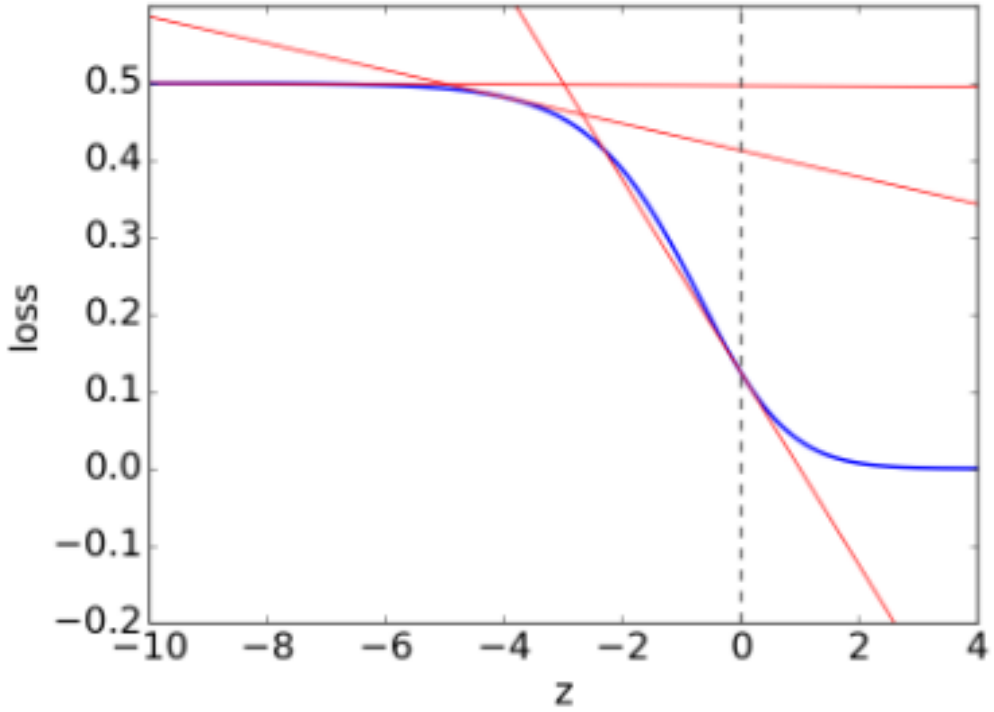
$$y = \sigma(z)$$ ← Función de activación

$$\mathcal{L}_{\text{SE}}(y, t) = \frac{1}{2}(y - t)^2.$$

Intento 3: El Problema

- El problema (asumiendo t = 1):

$$\frac{\partial \mathcal{L}}{\partial w_j} = \frac{\partial \mathcal{L}}{\partial z}\frac{\partial z}{\partial w_j}$$

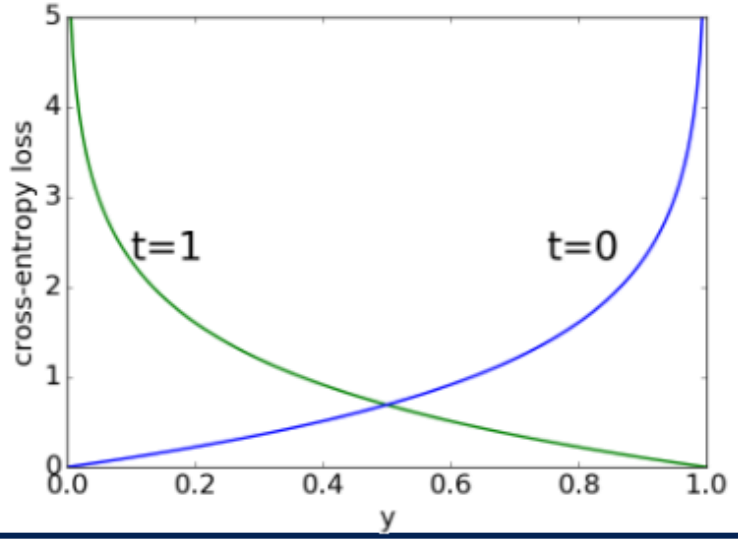


- Para $z \ll 0$, tenemos $\sigma(z) \approx 0$
- $\partial L/\partial z \approx 0 \implies \partial L/\partial w_j \approx 0 \implies$ la derivada respecto a w_j es pequeña
- w_j es como un punto crítico
- Si la predicción está muy mal, deberías estar lejos de un punto crítico (tu solución candidata).

Regresión Logística - Entropía Cruzada

- Como $y$siempre está entre 0 y 1, podemos pensar en $y$como **la probabilidad que el modelo predice de que la clase sea 1**.
  - Si $y = 0.9$, el modelo dice: "Estoy 90% seguro de que es clase 1"
  - Si $y = 0.2$, el modelo dice: "Estoy 80% seguro de que es clase 0"
- Los expertos que estaban 99% seguros de que Keiko ganaría estaban mucho más equivocados que aquellos que solo estaban 90% seguros.
- La **pérdida de entropía cruzada** (también conocida como pérdida logarítmica) captura esta intuición:

$$\mathcal{L}_{\mathrm{CE}}(y,t) = \begin{cases} -\log y & \text{if } t = 1 \\ -\log(1-y) & \text{if } t = 0 \end{cases}$$
$$= -t \log y - (1-t) \log(1-y)$$

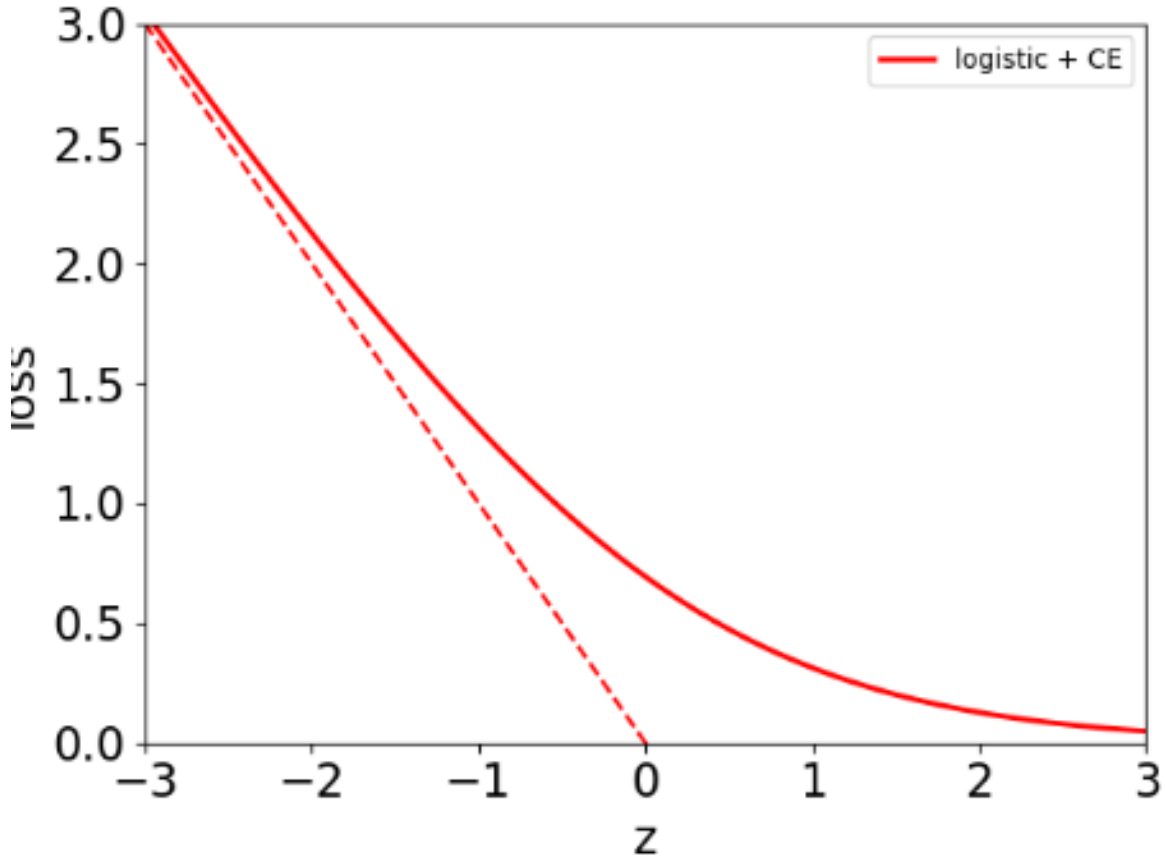


Regresión Logística

- Regresión Logística:

$$
\begin{aligned}
z &= \mathbf{w}^\top \mathbf{x} \\
y &= \sigma(z) \\
&= \frac{1}{1+e^{-z}} \\
\mathcal{L}_{\mathrm{CE}} &= -t \log y - (1-t) \log(1-y)
\end{aligned}
$$

- (El gráfico es para objetivo t = 1)



Regresión Logística - Inestabilidades Numéricas

- Si implementamos la regresión logística de manera ingenua, podemos tener inestabilidades numéricas.
- Considera: t = 1 pero estás muy seguro de que $z \ll 0$
- Si y es muy pequeño, puede ser **numéricamente cero**.
  - Esto causa bugs muy sutiles y difíciles de encontrar.

$$y = \sigma(z) \qquad \Rightarrow y \approx 0$$

$$\mathcal{L}_{\text{CE}} = -t \log y - (1-t) \log(1-y) \qquad \Rightarrow \text{ computes } \log 0$$

Regresión Logística - Versión Numéricamente Estable

- En su lugar, combinamos la función de activación y la pérdida en una sola función de **entropía-cruzada-logística**:

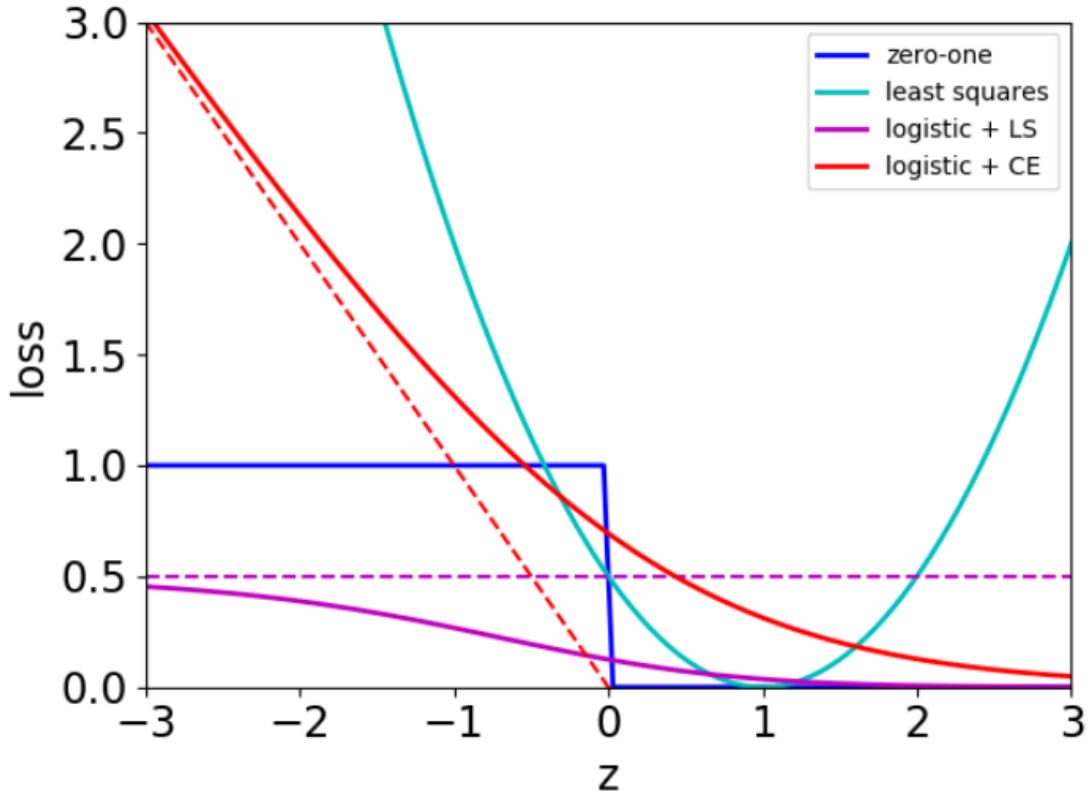
$$\mathcal{L}_{\mathrm{LCE}}(z,t) = \mathcal{L}_{\mathrm{CE}}(\sigma(z),t) = t\log(1+e^{-z}) + (1-t)\log(1+e^{z})$$

- Cálculo numéricamente estable:

```
E = t * np.logaddexp(0, -z) + (1-t) * np.logaddexp(0, z)
```

Comparación de Funciones de Pérdida

- Comparación de funciones de pérdida (para t = 1):



Descenso de Gradiente para Regresión Logística

- ¿Cómo minimizamos el costo J para regresión logística? No hay solución directa.
  - Tomar derivadas de J respecto a w e igualarlas a 0 no tiene solución explícita.
- Sin embargo, la pérdida logística es una función convexa en w, así que usamos descenso de gradiente.
  - Recordemos: inicializamos los pesos con algo razonable y ajustamos en la dirección de máximo descenso.
  - Una inicialización estándar es w = 0.

Gradiente de la Pérdida Logística

$$\mathcal{L}_{\mathrm{CE}}(y, t) = -\,t\log(y) - (1-t)\log(1-y)$$
$$y = 1/(1+e^{-z}) \quad \text{and} \quad z = \mathbf{w}^\top \mathbf{x}$$

- Por regla de la cadena:

$$\frac{\partial \mathcal{L}_{\mathrm{CE}}}{\partial w_j} = \frac{\partial \mathcal{L}_{\mathrm{CE}}}{\partial y} \cdot \frac{\partial y}{\partial z} \cdot \frac{\partial z}{\partial w_j} = \left(-\frac{t}{y} + \frac{1-t}{1-y}\right) \cdot y(1-y) \cdot x_j$$
$$= (y - t)x_j$$

- Actualización de descenso de gradiente:

$$w_j \leftarrow w_j - \alpha \frac{\partial \mathcal{J}}{\partial w_j}$$
$$= w_j - \frac{\alpha}{N} \sum_{i=1}^{N} (y^{(i)} - t^{(i)})\, x_j^{(i)}$$



Comparación de Actualizaciones

- Comparación de actualizaciones de descenso de gradiente:
- Regresión lineal:

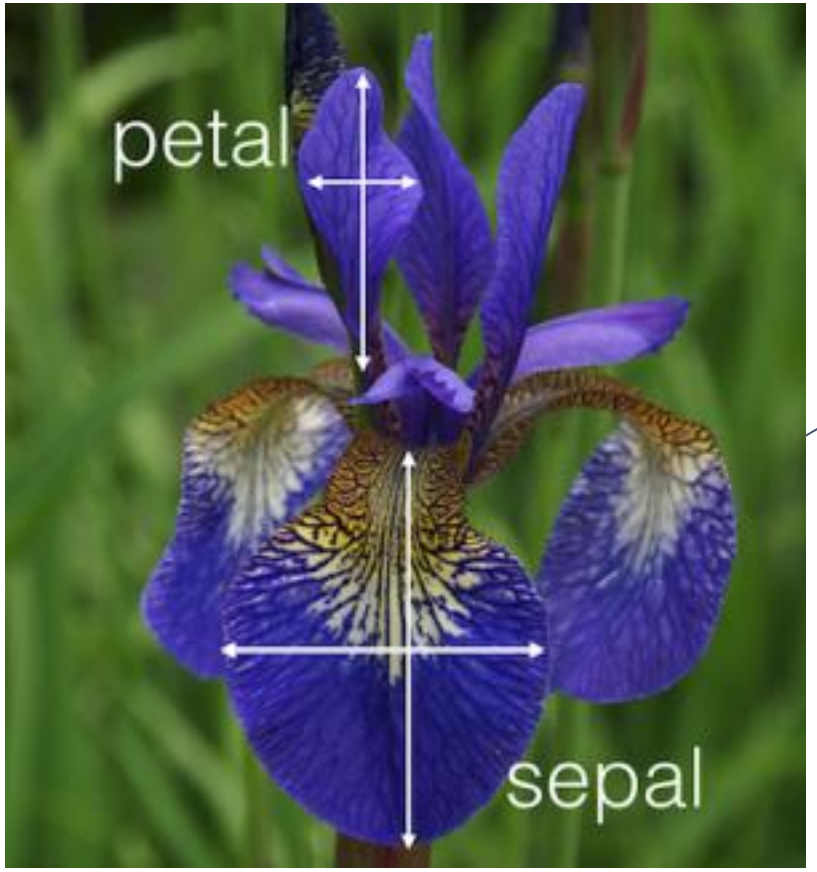
$$\mathbf{w} \leftarrow \mathbf{w} - \frac{\alpha}{N} \sum_{i=1}^{N} (y^{(i)} - t^{(i)})\, \mathbf{x}^{(i)}$$

- Regresión logística:

$$\mathbf{w} \leftarrow \mathbf{w} - \frac{\alpha}{N} \sum_{i=1}^{N} (y^{(i)} - t^{(i)})\, \mathbf{x}^{(i)}$$

- ¡No es coincidencia! Ambos son ejemplos de **modelos lineales generalizados**.

Ejemplo: Clasificación de iris



sepal_length	sepal_width	petal_length	petal_width
4.6	3.6	1.0	0.2
5.7	4.4	1.5	0.4
6.7	3.1	4.4	1.4
4.8	3.4	1.6	0.2
4.4	3.2	1.3	0.2

Target: class of iris plant

El Dataset



Iris setosa



Iris
versicolor



Iris
virginica



Objetivo

sepal_length	sepal_width	petal_length	petal_width
4.6	3.6	1.0	0.2

???

setosa!

~~versicolor~~

~~virginica~~

sepal_length	sepal_width	petal_length	petal_width
4.6	3.6	1.0	0.2

???

setosa!

Variables independientes:
[4x1]

3 resultados, indicando la probabilidad? de cada clase:

[1, 0, 0]
Setosa Versicolor Virginica
(one hot encoding)
[3x1]

Definición del modelo: Clasificador Lineal

sepal_length	sepal_width	petal_length	petal_width
4.6	3.6	1.0	0.2

Variables independientes:
[4x1]

data weights

f(**x**,**W**)

$$f(x, W) = Wx$$

setosa!

3 resultados, indicando la probabilidad? de cada clase:

[1, 0, 0]

Setosa Versicolor Virginica

(one hot encoding)

[3x1]

Definición del modelo: Clasificador Lineal

sepal_length	sepal_width	petal_length	petal_width
4.6	3.6	1.0	0.2



setosa!

$$f(x, W) = Wx$$

3xN **?** **4xN**

parameters, or “weights”

Definición del modelo: Clasificador Lineal

sepal_length	sepal_width	petal_length	petal_width
4.6	3.6	1.0	0.2

→ **setosa!**

$$f(x, W) = Wx$$

4xN

3xN **[3x4]**

parameters, or “weights”

Definición del modelo: Clasificador Lineal

sepal_length	sepal_width	petal_length	petal_width
4.6	3.6	1.0	0.2

→ **setosa!**

$$f(x, W) = Wx$$

3xN **[3x4]** **4xN**

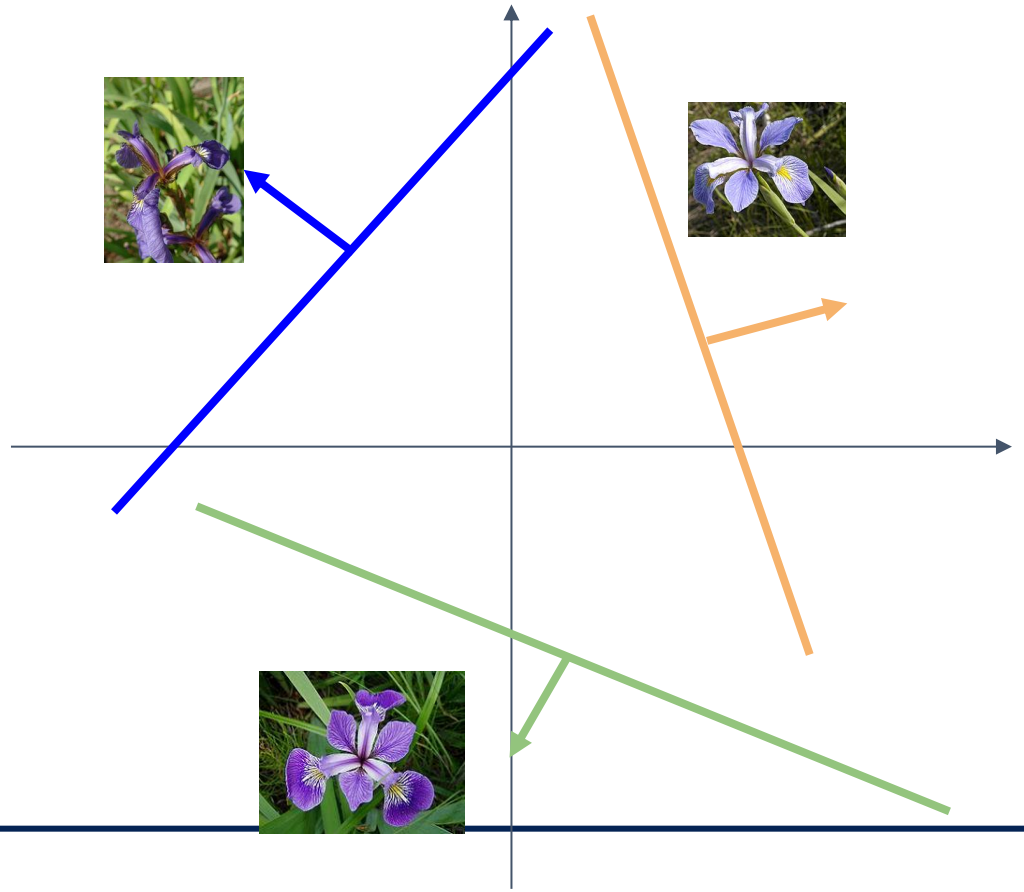
(+b) **3x1**

bias

parameters, or “weights”

Definición del modelo: Clasificador Lineal

$$f(x_i) = Wx_i + b$$

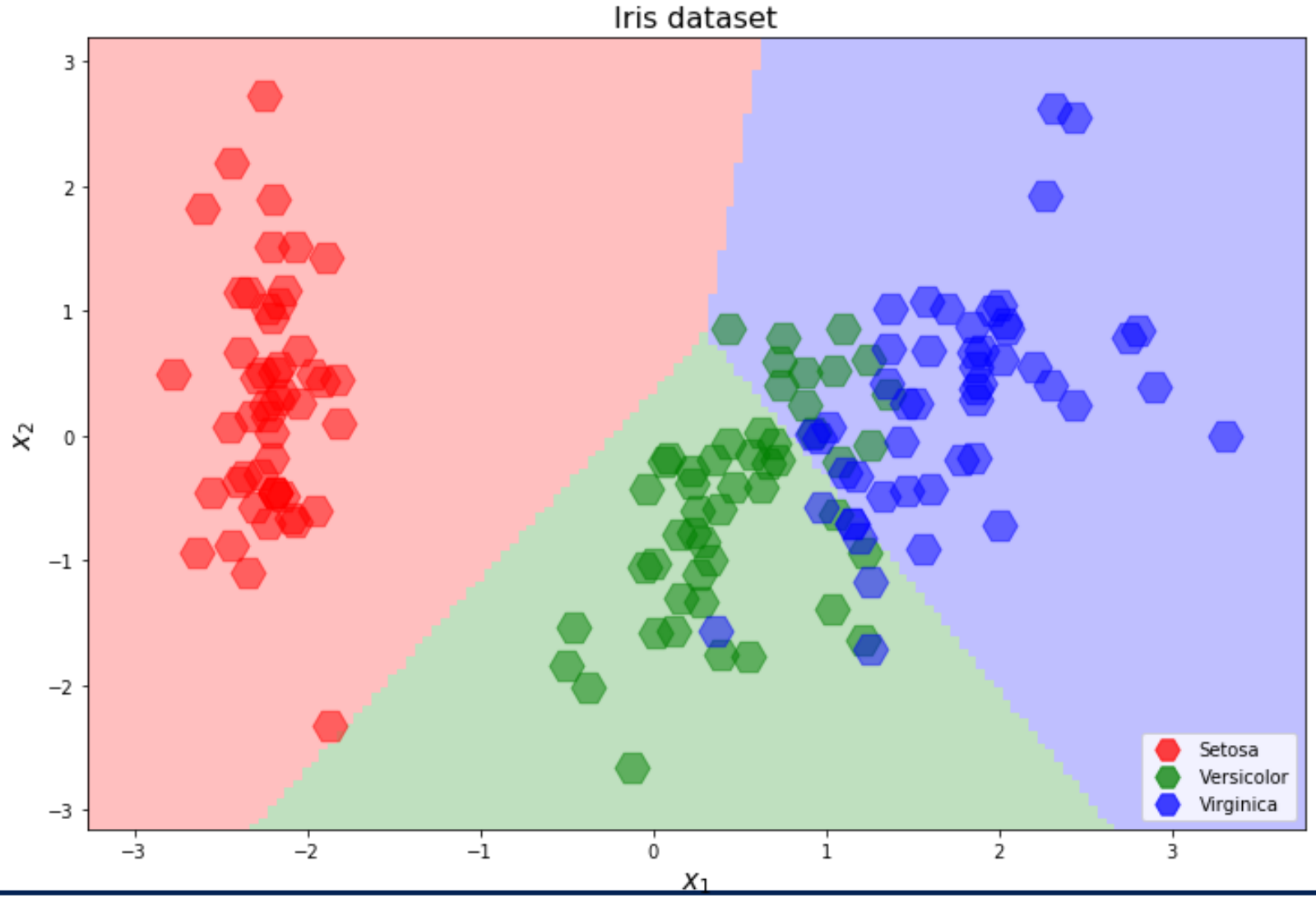


Setosa

Versicolor

Virginica

Interpretando el clasificador lineal



Ej:

sepal_length	sepal_width	petal_length	petal_width
4.6	3.6	1.0	0.2

setosa!

[1, 0, 0]

Setosa Versicolor

Virginica

X_i

4.6
3.6
1.0
0.2

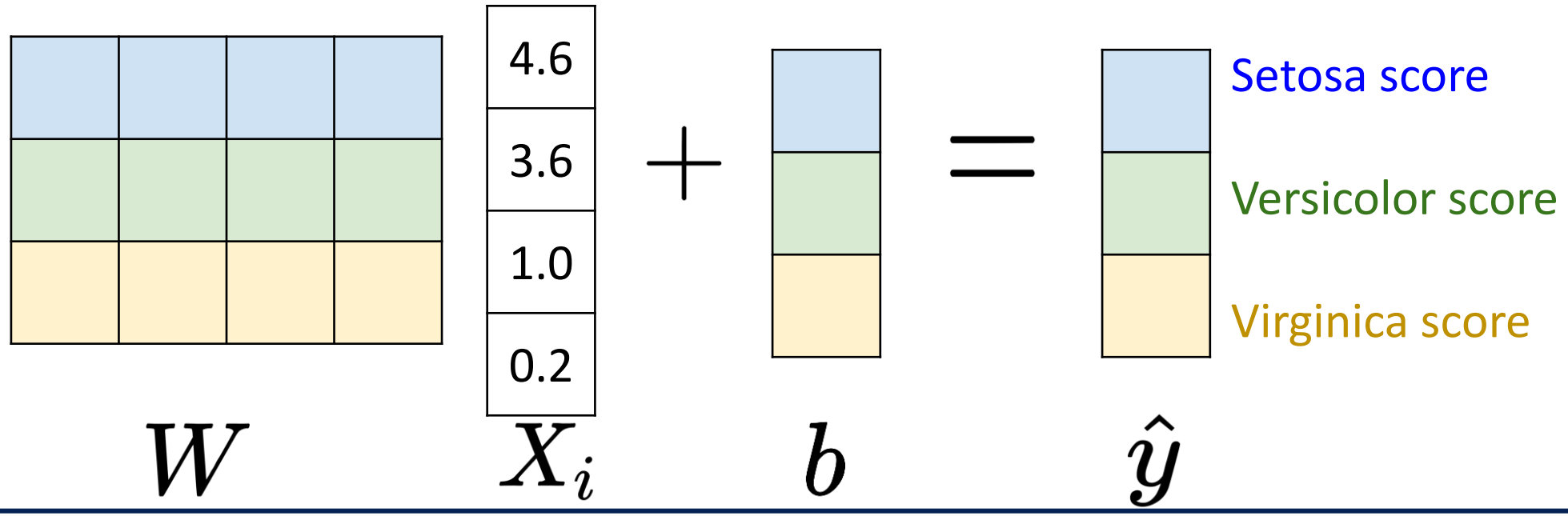
y_i

1
0
0

Ej:

sepal_length	sepal_width	petal_length	petal_width
4.6	3.6	1.0	0.2

$$\hat{y} = f(x_i) = Wx_i + b$$



Ej:

sepal_length	sepal_width	petal_length	petal_width
4.6	3.6	1.0	0.2

Random numbers!

$$\hat{y} = f(x_i) = Wx_i + b$$

-0.4	-0.1	-2.1	1.6
-1.8	-0.8	0.5	-1.2
-1.1	-0.9	0.6	2.3

W

4.6
3.6
1.0
0.2

X_i

$+$

1.8
0.4
0.1

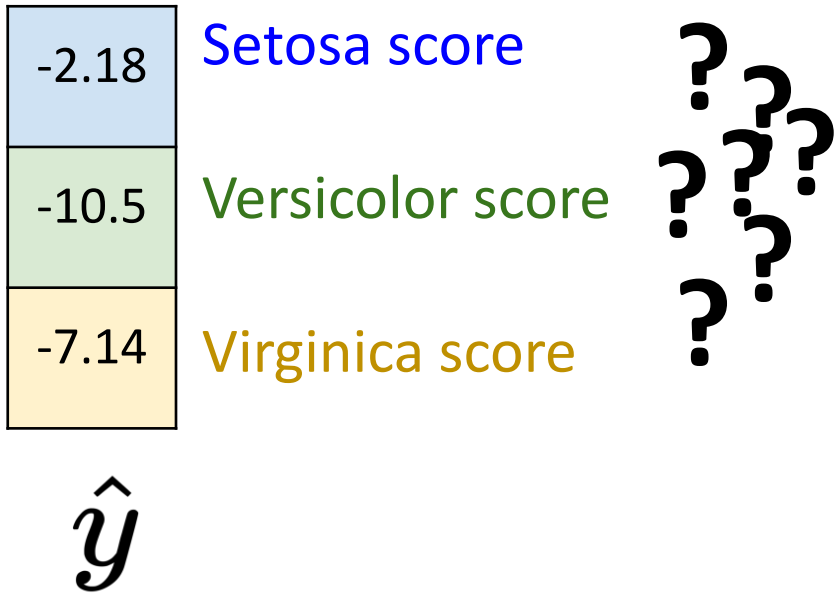
b

$=$

-2.18	Setosa score
-10.5	Versicolor score
-7.14	Virginica score

$\hat{y}$

¿Cómo interpretamos estos valores?





Softmax Classifier (Multinomial Logistic Regression)

-2.18	Setosa score
-10.5	Versicolor score
-7.14	Virginica score

$\hat{y}$

$$softmax(\hat{y}_i) = \frac{e^{\hat{y_i}}}{\sum_j e^{\hat{y_j}}}$$



Softmax Classifier (Multinomial Logistic Regression)

$$softmax(\hat{y}_i) = \frac{e^{\hat{y_i}}}{\sum_j e^{\hat{y_j}}}$$

-2.18	Setosa score
-10.5	Versicolor score
-7.14	Virginica score

$\hat{y}$



Softmax Classifier (Multinomial Logistic Regression)

$$softmax(\hat{y}_i) = \frac{e^{\hat{y}_i}}{\sum_j e^{\hat{y}_j}}$$

-2.18	Setosa score
-10.5	Versicolor score
-7.14	Virginica score

$\hat{y}$

exp →

0.11304
0.00003
0.00079

$e^{\hat{y}_i}$

Softmax Classifier (Multinomial Logistic Regression)

$$softmax(\hat{y}_i) = \frac{e^{\hat{y}_i}}{\sum_j e^{\hat{y}_j}}$$

-2.18	Setosa score
-10.5	Versicolor score
-7.14	Virginica score

$\hat{y}$

exp →

0.11304
0.00003
0.00079

$e^{\hat{y}_i}$

normalize →

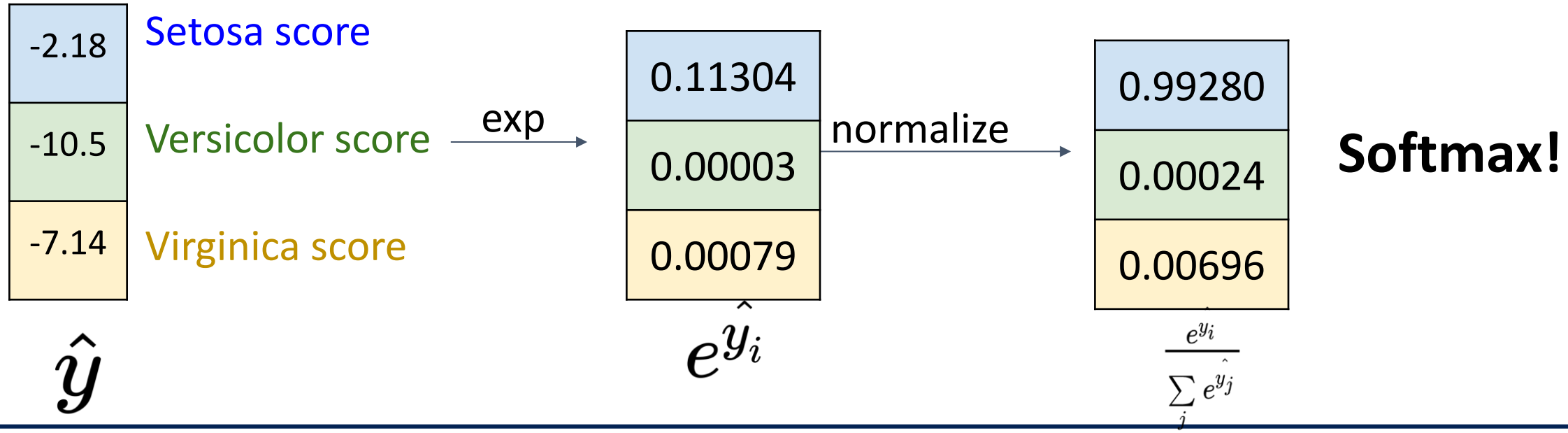
0.99280
0.00024
0.00696

$\frac{e^{\hat{y}_i}}{\sum_j e^{\hat{y}_j}}$

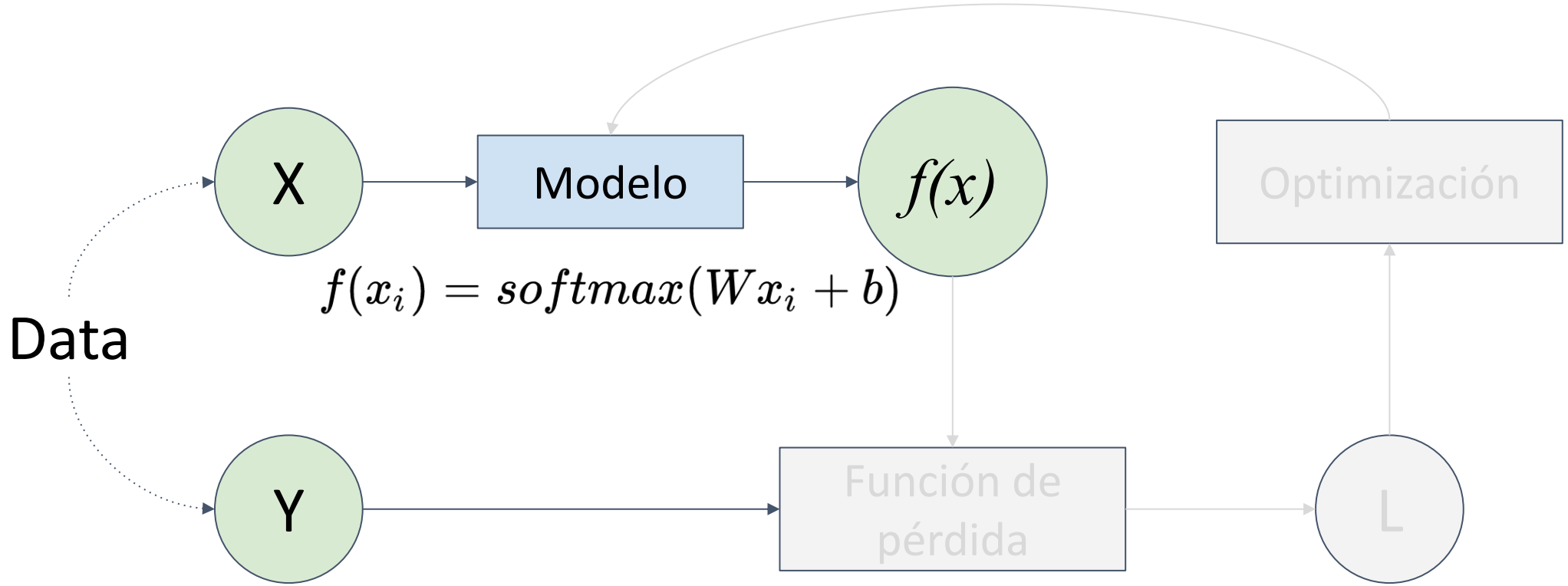
Softmax!

Definición final del modelo

$$f(x_i) = softmax(Wx_i + b)$$

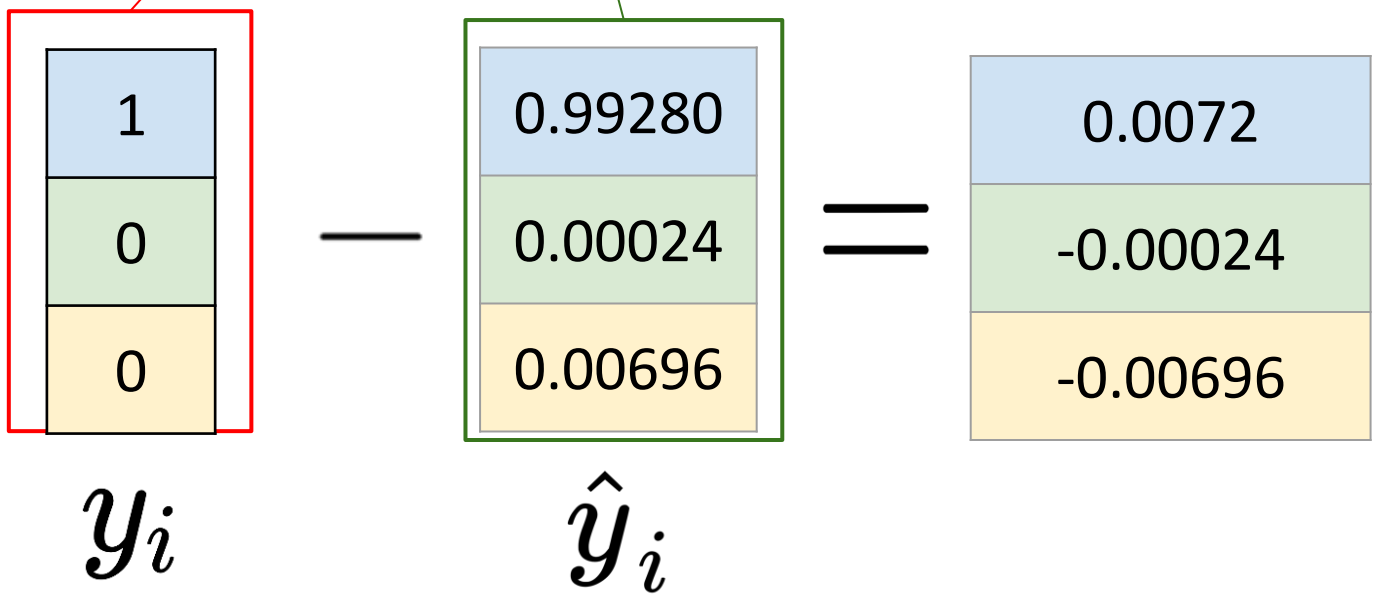


Regresión Logística



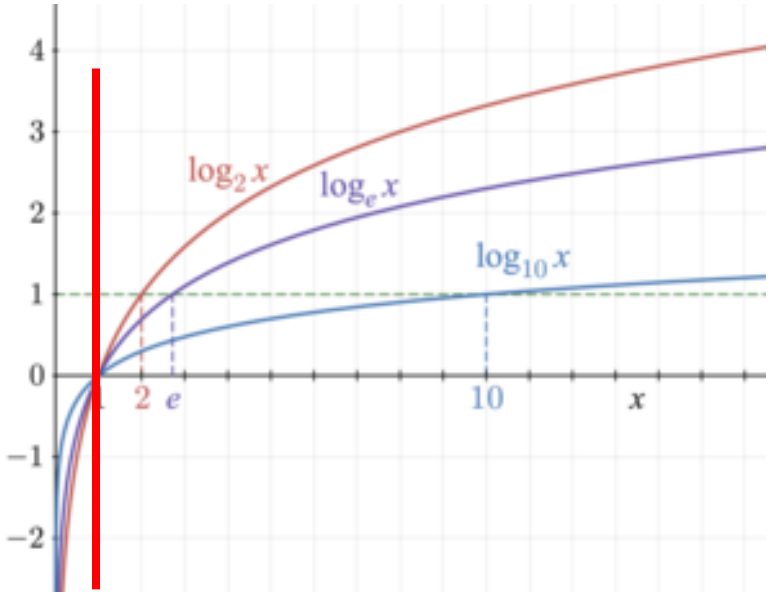
Elegir una función de pérdida

$$L = \frac{1}{n}\sum_{i=1}^{n}(Y_i - \hat{Y}_i)^2$$



Cross Entropy (Función de pérdida)

$$L = -\sum_{i=1}^{m} y_i log(\hat{y}_i)$$

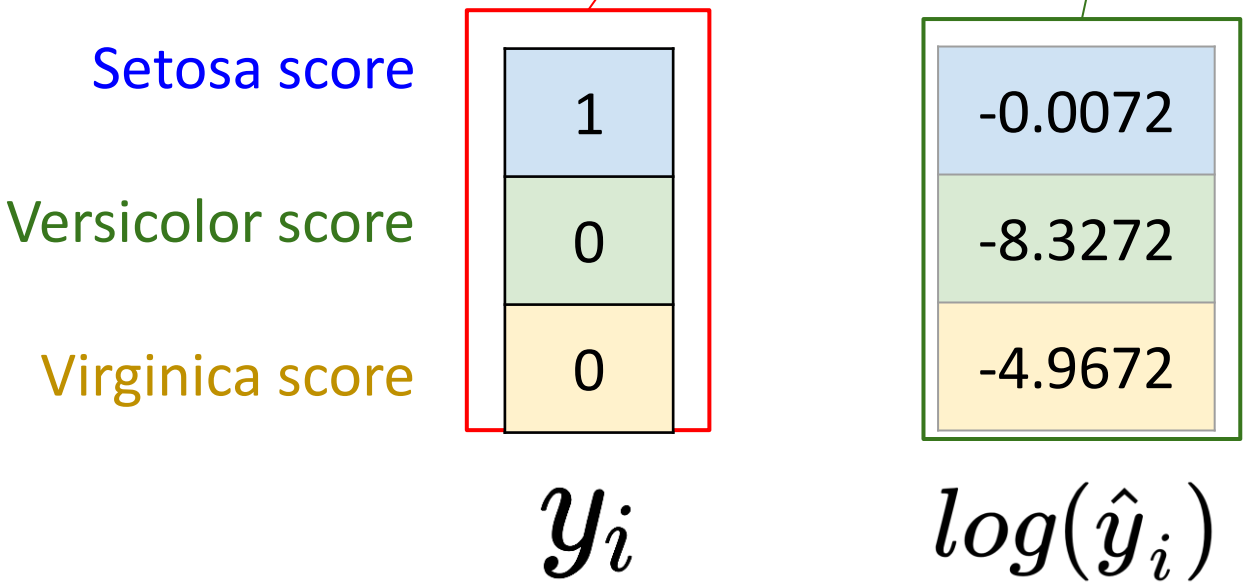


y_i
1
0
0

$\hat{y}_i$
0.99280
0.00024
0.00696

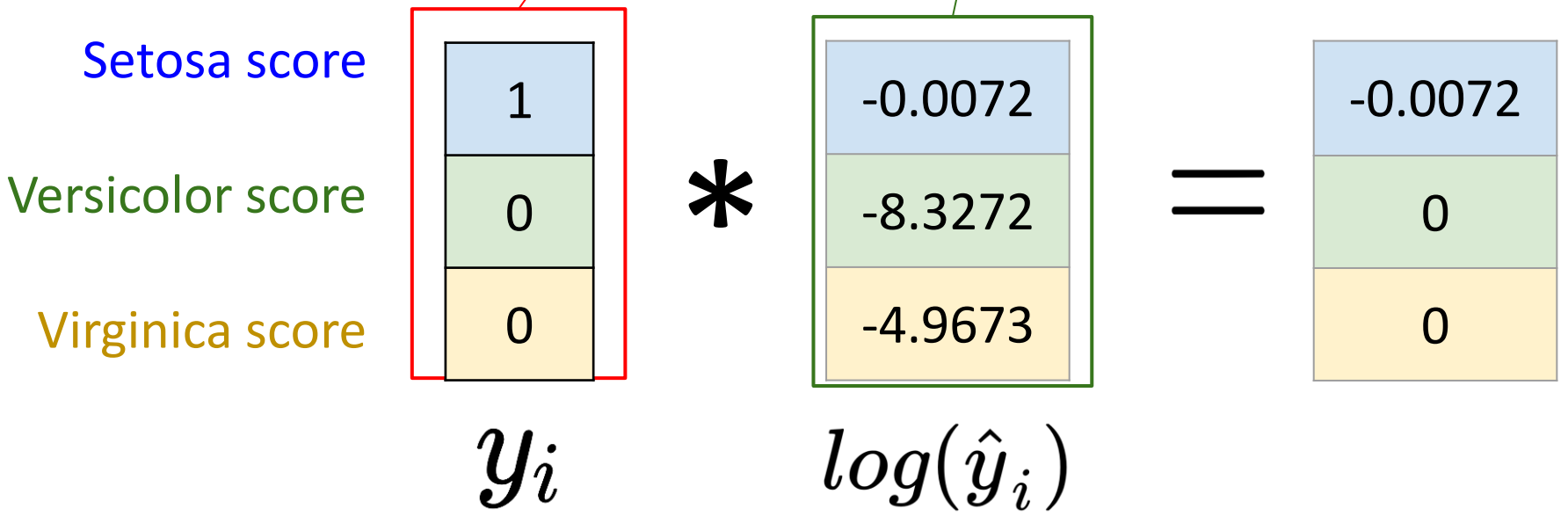
Cross Entropy (Función de pérdida)

$$L = -\sum_{i=1}^{m} y_i \, log(\hat{y}_i)$$



Cross Entropy (Función de pérdida)

$$L = -\sum_{i=1}^{m} y_i \, log(\hat{y}_i)$$



Cross Entropy (Función de pérdida)

$$L = -\sum_{i=1}^{m} y_i log(\hat{y}_i)$$

$$L = -\sum \begin{matrix} -0.0072 \\ 0 \\ 0 \end{matrix} = \mathbf{0.00723}$$



Entropy

$$-\sum p(x) \log p(x)$$

Cross-entropy

$$-\sum p(x) \log q(x)$$

Cross Entropy (Función de pérdida)

$$L = -\sum_{i=1}^{m} y_i log(\hat{y}_i) = -\sum$$

-0.0072
0
0

Setosa score

Versicolor score

Virginica score

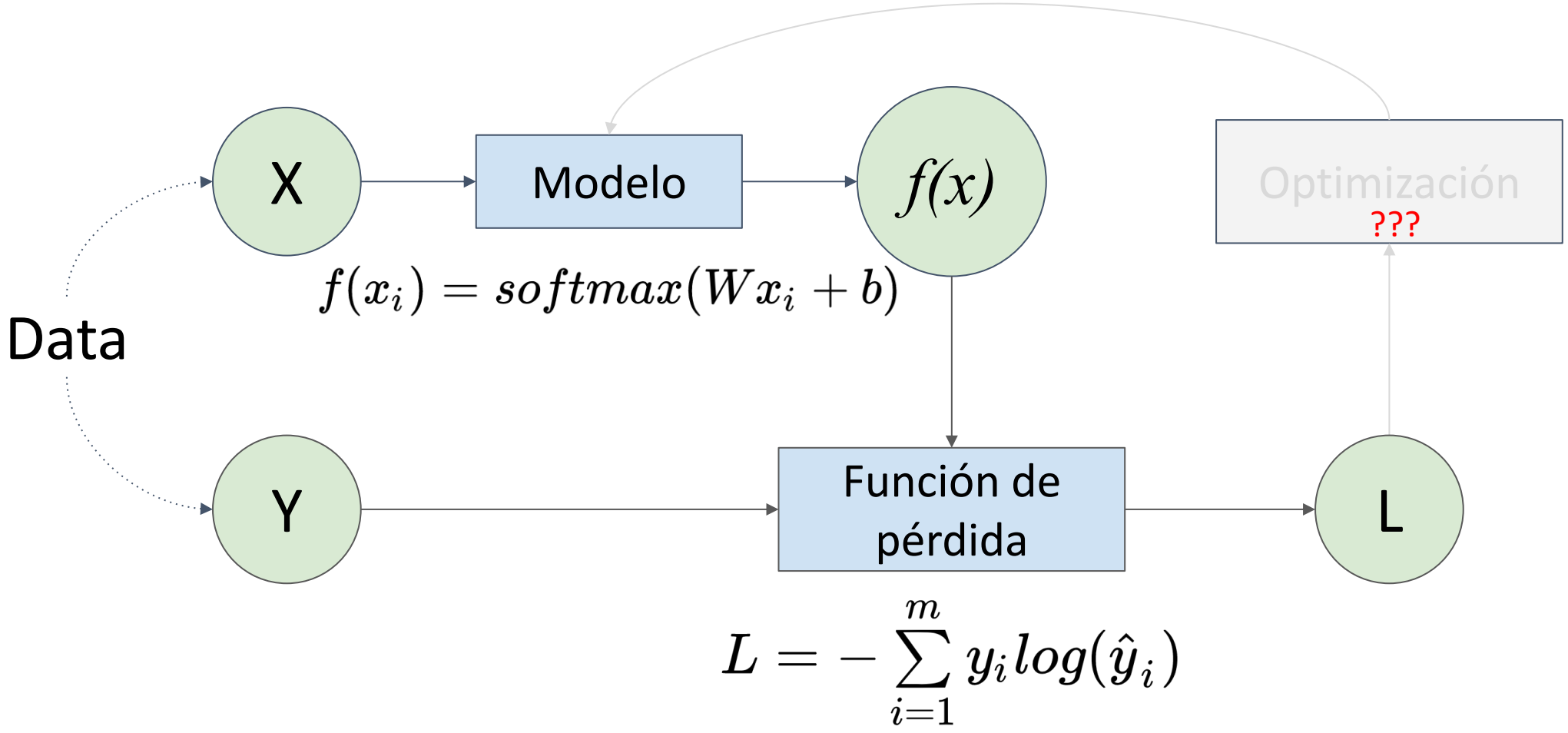
y_i
1
0
0

$\hat{y}_i$
0.99280
0.00024
0.00696

$=$ **0.0072**

Si usamos la predicción del modelo, 0.0072 representaría la información que obtendríamos al conocer la data real.

Regresión Logística



Problema 1: Actualización de la gradiente

Problema: Considere el siguiente conjunto de datos con 3 puntos (Peso, Altura):

$$(0.5, 1.4), \quad (2.3, 1.9), \quad (2.9, 3.2)$$

Deseamos ajustar una línea de regresión lineal $y = mx + b$ a estos datos. Suponga que la **pendiente (m) está fija en 0.64** (estimación por mínimos cuadrados) y queremos encontrar el **intercepto (b)** óptimo utilizando el algoritmo de Descenso de Gradiente.

Parámetros iniciales:

- Intercepto inicial (b_{old}): **0**
- Tasa de aprendizaje (Learning Rate): **0.1**

Realice **una (1) iteración** completa del algoritmo de Descenso de Gradiente para actualizar el valor del intercepto. Muestre el cálculo de la derivada, el tamaño del paso (step size) y el nuevo intercepto (b_{new}). Utilice la **Suma de los Residuos al Cuadrado (SSR)** como función de pérdida.



Paso 1: Definir la Función de Pérdida y su Derivada

La función de pérdida es la Suma de los Residuos al Cuadrado (SSR):

$$SSR = \sum_{i=1}^{n}(y_i - \hat{y}_i)^2 = \sum_{i=1}^{n}(y_i - (mx_i + b))^2$$

Para optimizar el intercepto (b), derivamos la función con respecto a b (aplicando la regla de la cadena):

$$\frac{\partial SSR}{\partial b} = -2\sum_{i=1}^{n}(y_i - \hat{y}_i)$$

Paso 2: Calcular las Predicciones ($\hat{y}$) y Residuos

Usamos la ecuación actual de la línea: $\hat{y} = 0.64x + 0$.

1. **Dato 1 ($x = 0.5, y = 1.4$):**
    - Predicción: $0.64(0.5) + 0 = 0.32$
    - Residuo: $1.4 - 0.32 = \mathbf{1.08}$
2. **Dato 2 ($x = 2.3, y = 1.9$):**
    - Predicción: $0.64(2.3) + 0 = 1.472$
    - Residuo: $1.9 - 1.472 = \mathbf{0.428}$
3. **Dato 3 ($x = 2.9, y = 3.2$):**
    - Predicción: $0.64(2.9) + 0 = 1.856$
    - Residuo: $3.2 - 1.856 = \mathbf{1.344}$

Paso 3: Calcular el Gradiente (La Pendiente de la Curva de Error)

Sumamos los residuos calculados en el paso anterior:

$$\text{Suma de Residuos} = 1.08 + 0.428 + 1.344 = \mathbf{2.852}$$

Ahora aplicamos la fórmula de la derivada:

$$\frac{\partial SSR}{\partial b} = -2 \times (2.852) = \mathbf{-5.704}$$



Paso 4: Calcular el Tamaño del Paso (Step Size)

Multiplicamos el gradiente por la tasa de aprendizaje (Learning Rate):

$$\text{Step Size} = \text{Gradiente} \times \text{Learning Rate}$$

$$\text{Step Size} = -5.7 \times 0.1 = \mathbf{-0.57}$$

Paso 5: Actualizar el Intercepto

Restamos el tamaño del paso al valor original (recuerda: restar un número negativo es sumar):

$$b_{new} = b_{old} - (\text{Step Size})$$

$$b_{new} = 0 - (-0.57)$$

$$\mathbf{b_{new} = 0.57}$$

Problema 2: Desvanecimiento de gradiente

Contexto: Tenemos un modelo de Regresión Logística simple: $\hat{y} = \sigma(z)$, donde $z = w \cdot x + b$. Queremos que el modelo aprenda a predecir la clase **1** ($y = 1$), pero el modelo actualmente está muy seguro de que la respuesta es **0** (está "saturado" en el error).

$$L = \frac{1}{2}(y - \hat{y})^2$$

Datos y Parámetros:

- Dato de entrada (x): **2**
- Etiqueta real (y): **1**
- Peso actual (w): **-2**
- Sesgo actual (b): **-1**

1. Calcule el valor lineal z y la predicción $\hat{y}$ (use la aproximación $\sigma(-5) \approx 0.007$).
2. Calcule la **derivada de la función Sigmoide** $\sigma'(z)$ en ese punto.
3. Calcule el **gradiente del error respecto al peso** ($\frac{\partial L}{\partial w}$) usando la Regla de la Cadena.
4. **Pregunta conceptual:** Compare el tamaño del Error $(y - \hat{y})$ con el tamaño del Gradiente calculado. ¿Qué observa?



Paso 1: Calcular la Predicción Actual (Forward Pass)

Primero, calculamos la salida lineal z y aplicamos la función de activación Sigmoide.

$$z = w \cdot x + b = (-2)(2) + (-1) = -4 - 1 = -5$$

Aplicamos la sigmoide (la neurona está saturada en la zona negativa):

$$\hat{y} = \sigma(-5) = \frac{1}{1 + e^{5}} \approx \mathbf{0.007}$$

Paso 2: Calcular la Derivada de la Sigmoide

La fórmula de la derivada de la sigmoide es $\sigma'(z) = \hat{y}(1 - \hat{y})$. Este es el término clave que causa el desvanecimiento.

$$\sigma'(-5) = 0.007 \times (1 - 0.007)$$

$$\sigma'(-5) = 0.007 \times 0.993 \approx \mathbf{0.0069}$$

Paso 3: Calcular el Gradiente (Backward Pass)

Usamos la Regla de la Cadena para hallar el gradiente de la pérdida respecto al peso w:

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w}$$

Desglosamos los términos:

1. Derivada del Error (MSE): $-(y - \hat{y}) = -(1 - 0.007) = \mathbf{-0.993}$
2. Derivada de la Activación: $\sigma'(z) = \mathbf{0.0069}$ (del Paso 2)
3. Derivada de la Entrada: $x = \mathbf{2}$

Multiplicamos todo:

$$\text{Gradiente} = (-0.993) \times (0.0069) \times (2)$$

$$\text{Gradiente} \approx \mathbf{-0.0137}$$



Paso 4: Análisis (Respuesta Conceptual)

- **Error del Modelo:** El error es enorme. La diferencia $(y - \hat{y})$ es **0.993**. Al modelo le falta casi todo el camino para llegar a la respuesta correcta.
- **Gradiente Calculado:** El gradiente es **-0.0137**.

Conclusión: A pesar de que el modelo tiene un error gigantesco (casi 100% incorrecto), el gradiente es minúsculo (casi 0). Si usamos una tasa de aprendizaje estándar (ej. 0.1), el cambio en el peso sería:

$$\Delta w = -(0.1) \times (-0.0137) = 0.00137$$

El peso cambiaría de -2 a -1.9986. ¡A este ritmo tardaría una eternidad en corregirse!

Este es el **Desvanecimiento del Gradiente**: La derivada de la función de activación (σ') es tan pequeña en la zona de saturación que "mata" la señal de error, impidiendo que los pesos se actualicen, aunque el modelo esté completamente equivocado.

Preguntas Vibe Quest

- La solución directa de regresión lineal usa $(X^TX)^{-1}X^Tt$, mientras que el descenso de gradiente es iterativo. Crea una simulación que muestre ambos métodos llegando al mismo resultado. ¿A partir de cuántas dimensiones (D) el descenso de gradiente se vuelve más eficiente que invertir la matriz?
- Crea un “laboratorio de mapeo de características” donde puedas probar diferentes bases: polinomiales, trigonométricas (senos/cosenos), y gaussianas. ¿Cómo afecta la elección de la base a la forma de la función ajustada? Encuentra una combinación que capture patrones periódicos en datos sintéticos
- La función sigmoide $\sigma(z) = 1/(1+e^{-z})$ transforma valores reales a “probabilidades”. Construye un visualizador interactivo que muestre cómo cambia la frontera de decisión de un clasificador logístico mientras ajustas los pesos w y el sesgo b. ¿Cómo se relaciona la magnitud de w con la 'confianza' del clasificador? Utiliza un análisis bayesiano para responder.
- Softmax generaliza la sigmoide a múltiples clases, convirtiendo scores arbitrarios en probabilidades que suman 1. ¿Cómo funciona esta 'competencia' entre clases?¿Se podría considerar probabilidad o no? Utiliza un análisis bayesiano para responder.